



人工智能：搜索技术 I



授课对象：计算机科学与技术专业 二年级

课程名称：人工智能（专业必修）

节选内容：第四章 搜索技术 I

课程学分：3学分



搜索定义



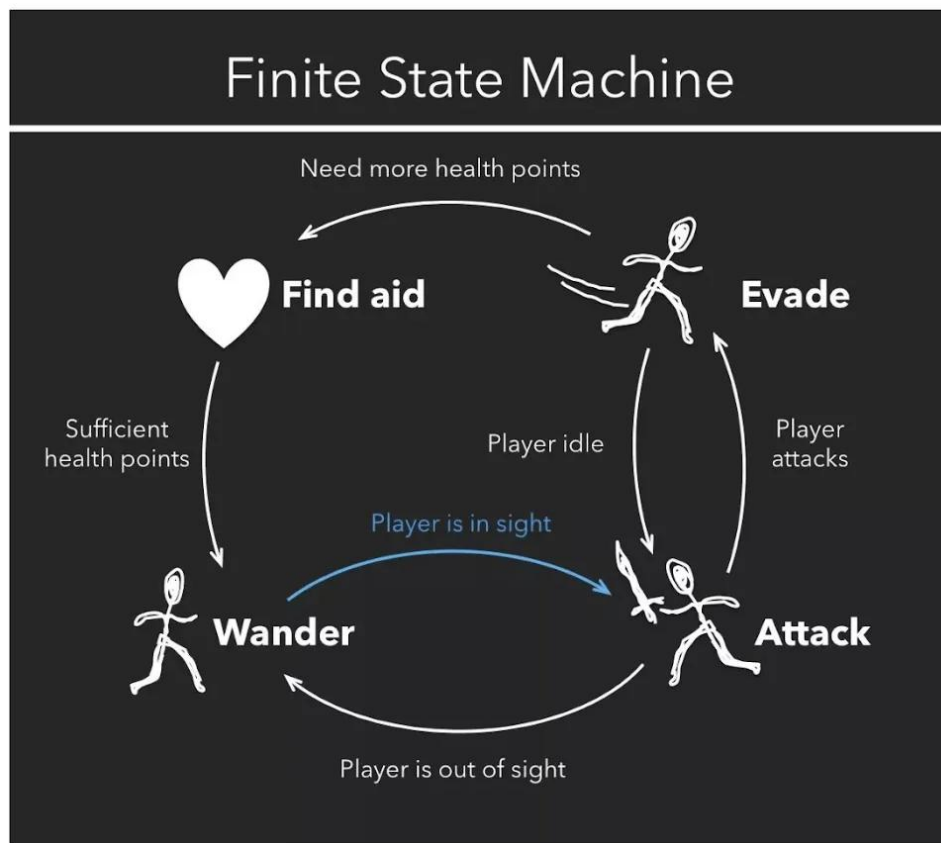
搜索Search

- Problem solving by search 搜索可以解决的问题
- Uninformed search 盲目（无信息）搜索
- Heuristic search 启发式（有信息）搜索



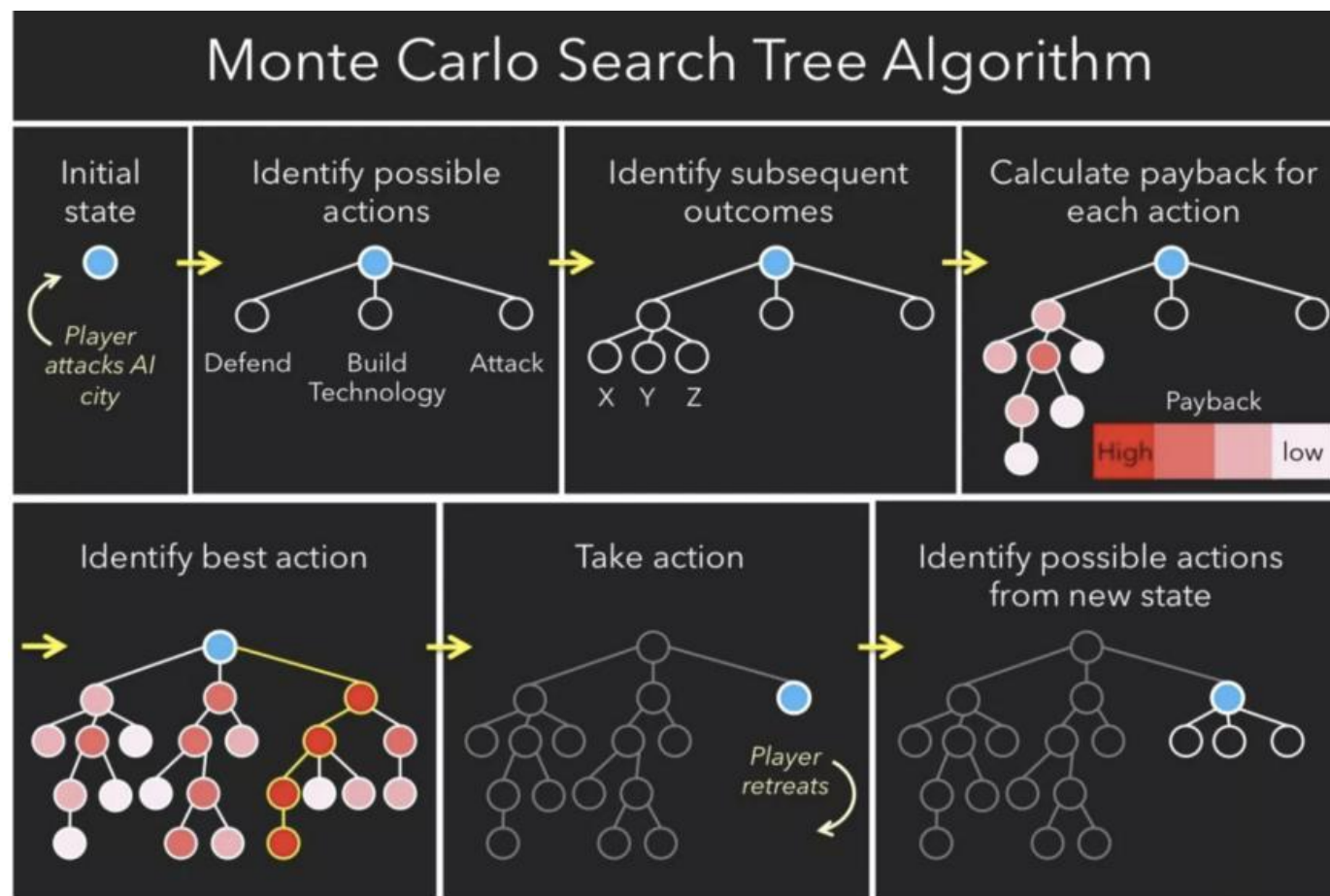
搜索Search

- 搜索在游戏策略中取得了较好的结果
- 其它人工智能问题中也可以应用搜索算法





搜索Search





搜索Search

- 搜索在游戏策略中取得了较好的结果
- 其它人工智能问题中也可以应用搜索算法

Practical

- Many problems don't have specific algorithms for solving them. Casting as search problems is often the easiest way of solving them.
- Problem specific heuristics provides search with a way of exploiting extra knowledge.



搜索Search

Search Problems

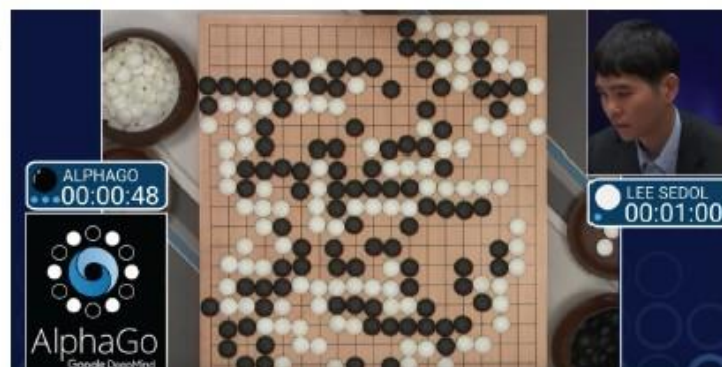


Slide 7



搜索Search

More search problems





搜索Search

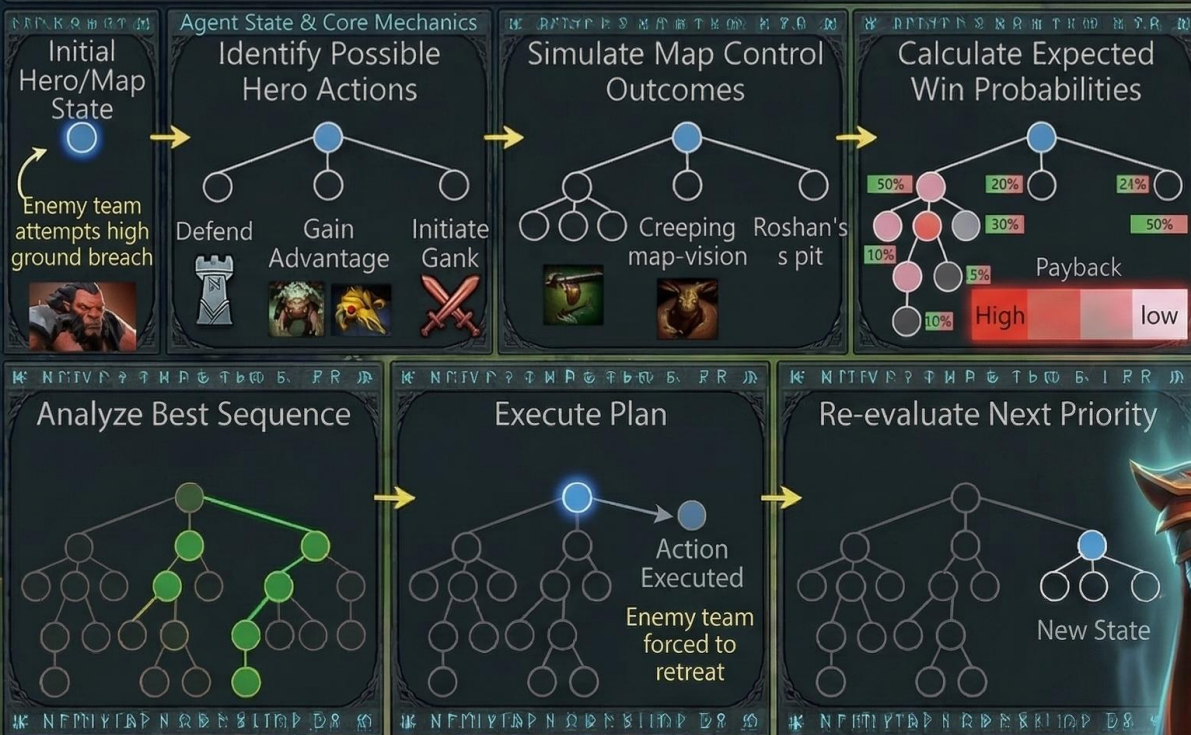
A Search Problem: How do Agent make decisions?

- This kind of hypothetical reasoning involves asking
 - what state am I ? 我在哪?
 - what actions can I take? 我能干吗?
 - what state should I achieve? 我到哪里去?
- From this we can reason about particular sequences of actions one should try to bring about to achieve a desirable state.
- Search is a computational method for capturing a particular version of this kind of schedule.



搜索Search

Monte Carlo Search Tree Algorithm



— what state am I ?

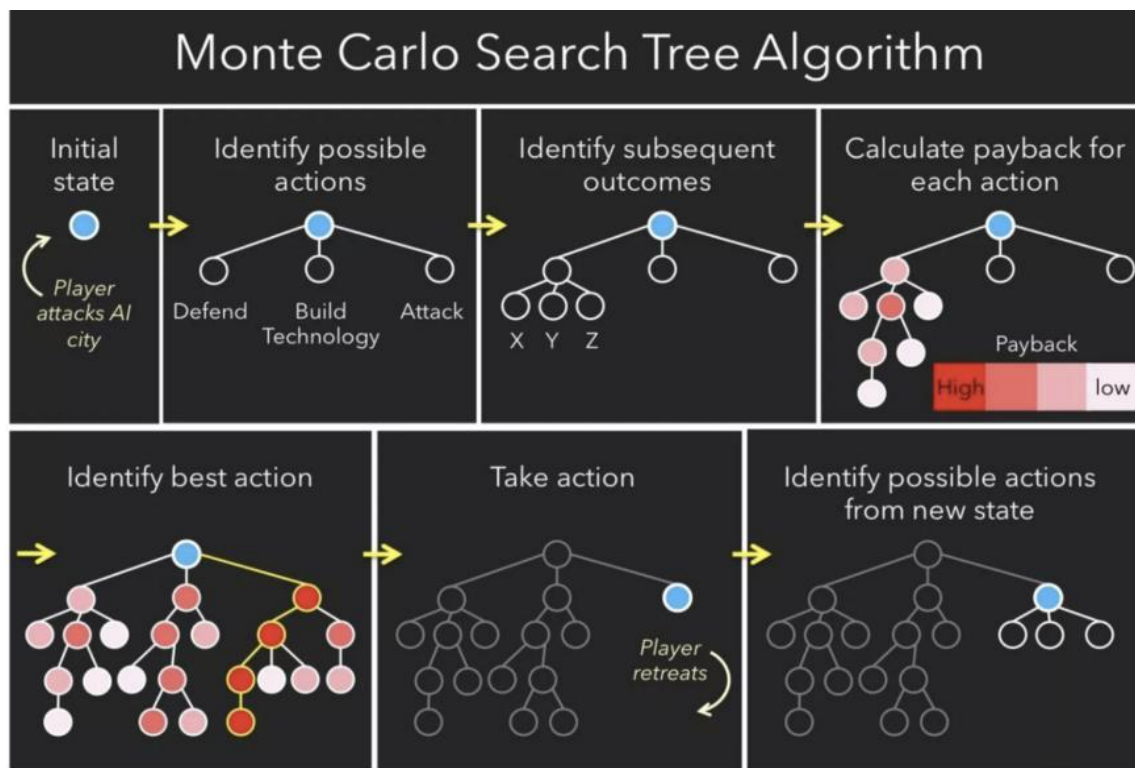
Current Map Control,
Itemization, and Ability
Cooldowns

— what actions can I take?

Defend, Farm, Gank, or
Force Teamfight



搜索Search



- what state am I ?
Current resources
- what actions can I take?
Defend/Build Tech/Attack
- what state should I achieve?
Defeat the player

Limitation of Search: Search only shows how to solve the problem once we have the **problem correctly formulated**.



形式化定义

我们需要考虑下面几个部分来对搜索问题进行形式化定义：

- **状态空间(state space)**: 表示需要进行搜索的空间。状态空间是对**问题**的形式化
- **动作(action)**: 表示从一个状态到另一个状态。动作是对**真实的动作**的形式化
- **初始状态(initial state)**: **当前状态**的表示
- **目标(goal)**: 需要达到的**目标状态**的表示
- **启发式方法(heuristics)**: 用于指导搜索的**前进方向**
- **解(solution)**: 是由 “**动作**” 构成的**序列**



形式化定义

Once you have a formalized search problem, there are a number of algorithms one can use to solve it.

A solution is a sequence of actions or moves that can transform your current state into a state where desired (or goal) conditions hold. 问题的解是由“动作”构成的序列。序列中的动作可以将初始状态转化为目标状态。

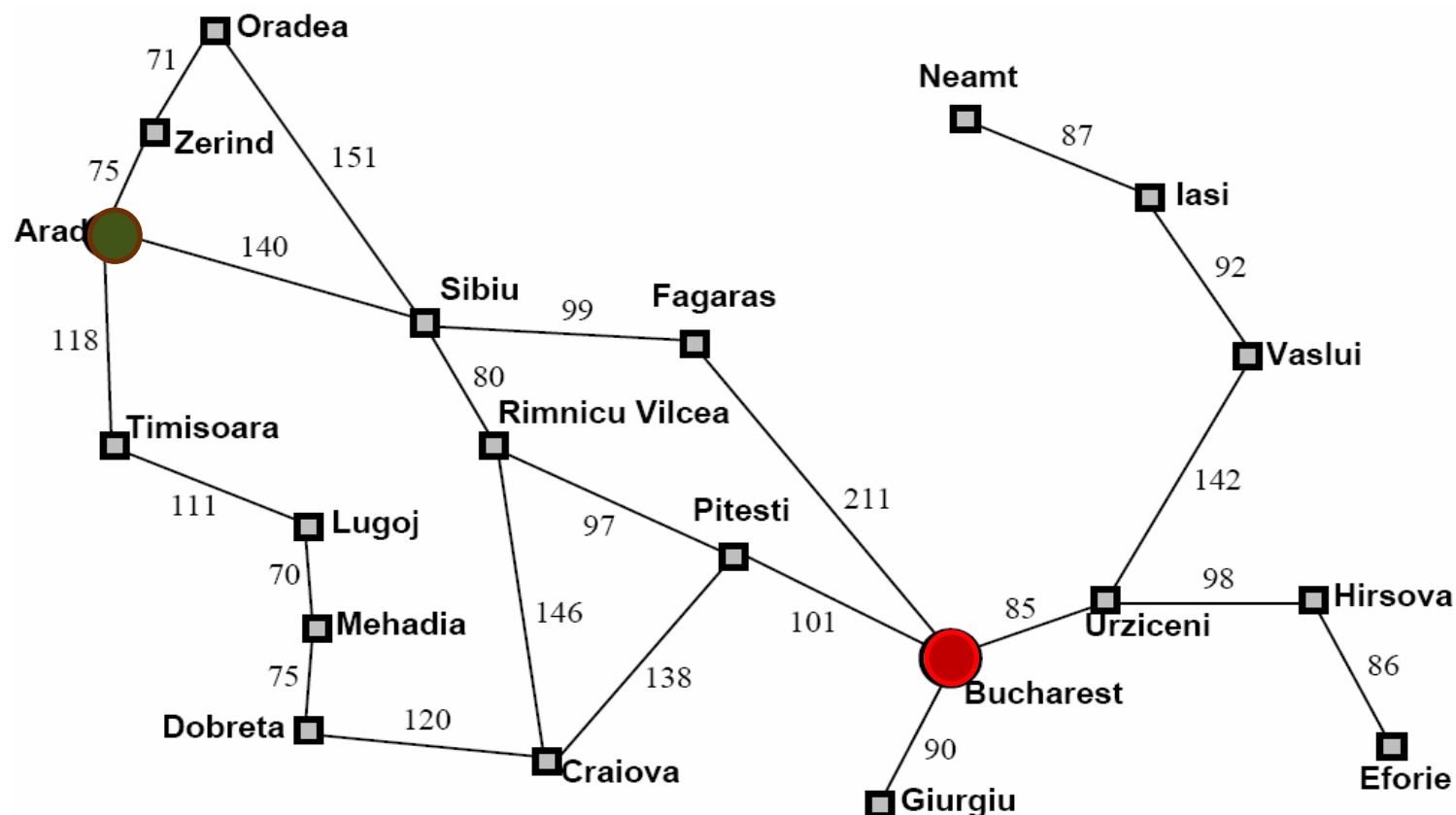




例1: Romania Travel

Currently in **Arad**, need to get to **Bucharest** ASAP.

Can we formalize this search?



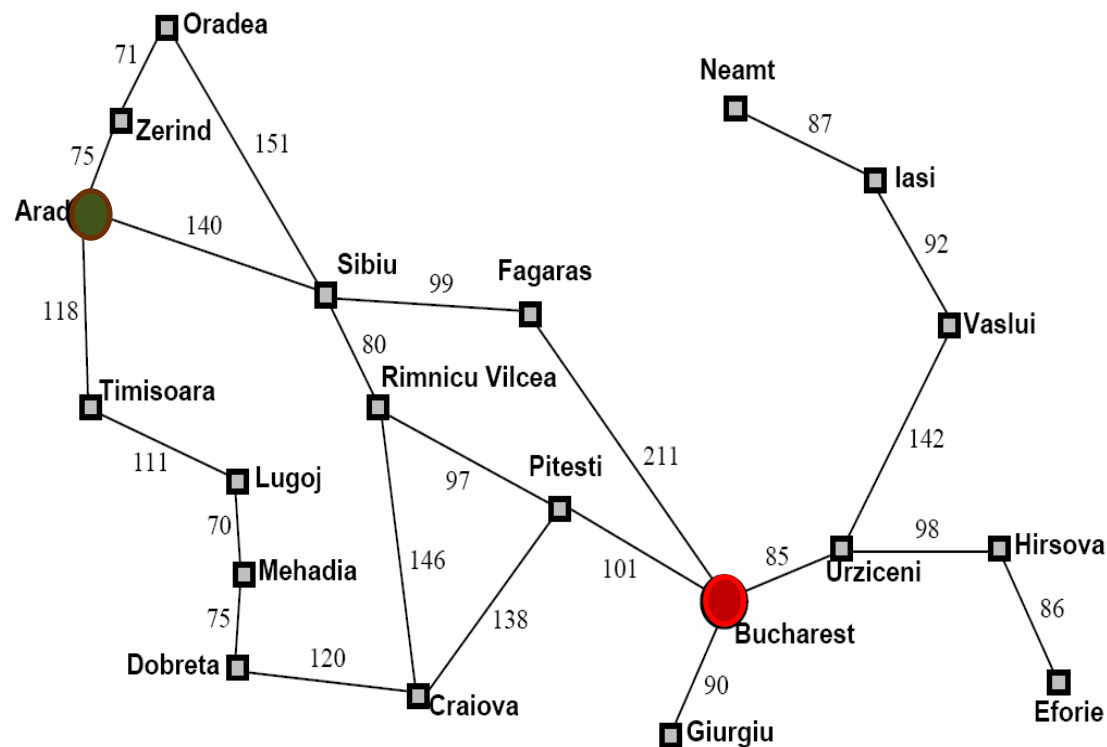
- state space:
- actions (successor functions):
- initial state:
- desired (or goal) condition:



例1: Romania Travel

Currently in **Arad**, need to get to **Bucharest** ASAP.

Can we formalize this search?



- state space
可以到达的任一个城市
- actions
在相邻的城市之间移动
- initial state
在城市Arad
- desired (or goal) condition
在城市Bucharest

问题的解：旅行路线，即从**Arad**到**Bucharest**途径城市组成的序列



例2: Water Jugs

我们有一个3加仑（升）的量杯和一个4加仑的量杯。我们可以从水龙头将任何一个量杯倒满，可以清空任何一个量杯，或者可以将一个量杯的水倒入另一个量杯（直到另一个量杯倒满为止）。

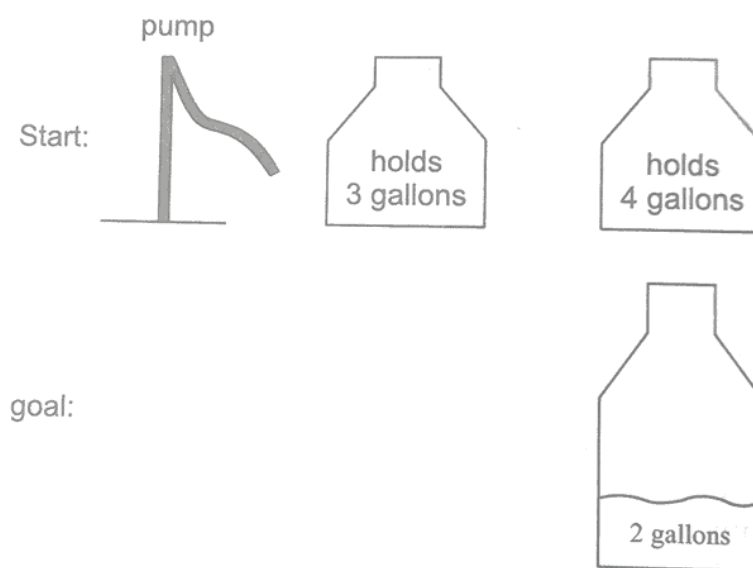


Fig: Water Jug Problem



例2: Water Jugs

–state space:

pairs of numbers (gal3, gal4) where gal3 is the number of gallons in the 3 gallon jug, and gal4 is the number of gallons in the 4 gallon jug.

–actions (successor functions):

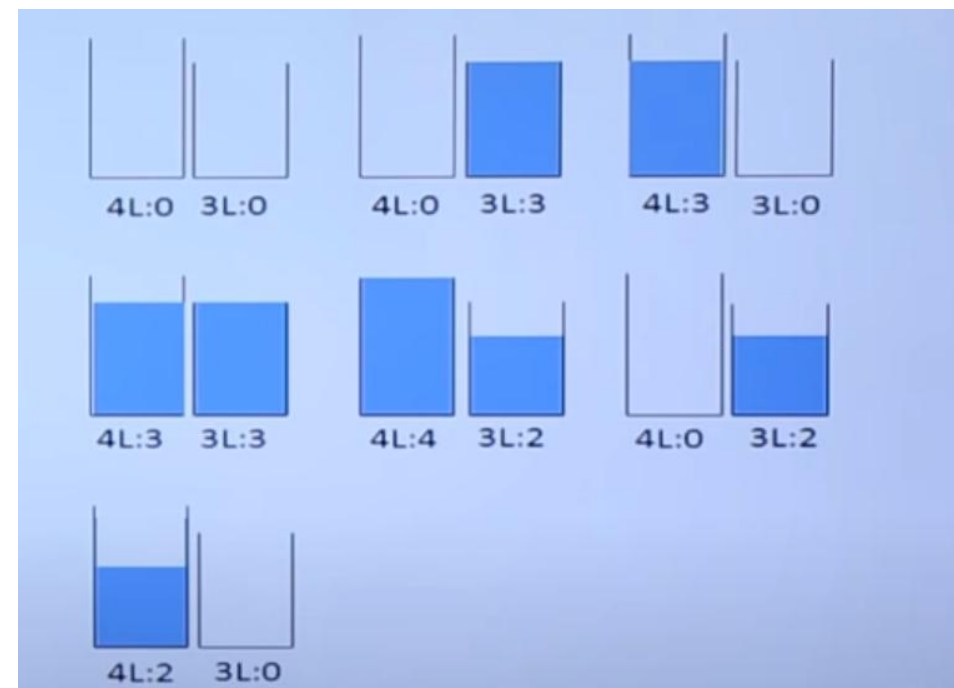
Empty-3-Gallon, Empty-4- Gallon, Fill-3-Gallon, Fill-4-Gallon, Pour-3-into-4, Pour 4-into-3.

–initial state:

Various, e.g., (0,0)

–desired (or goal) condition:

Various, e.g., (0,2) or (1, 3)



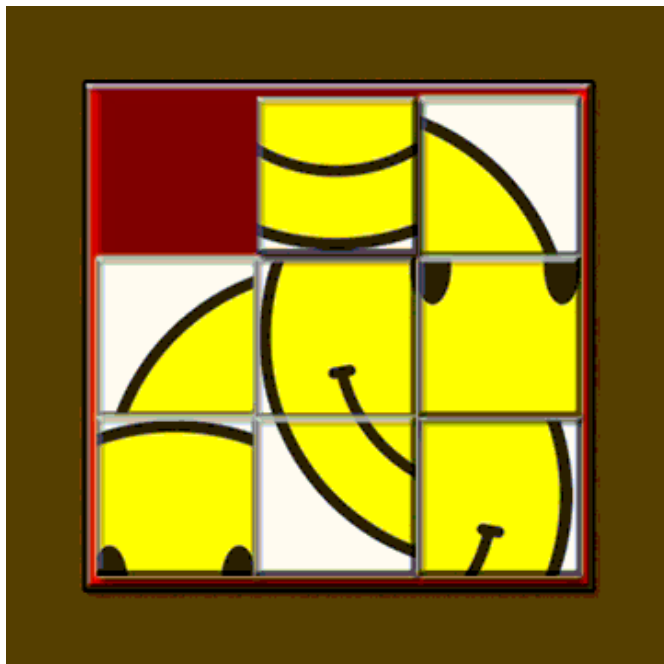


例2: Water Jugs

- If we start off with gal3 and gal4 as integers, can only reach integer values.
- Some values, e.g., (1,2) are not reachable from some initial state, e.g., (0,0).
- Some actions do not change the state, e.g.,
 - $(0,0) \rightarrow \text{Empty-3-Gallon} \rightarrow (0,0)$



例3: The 8-Puzzle/八数码问题



规则: 可以把空格移动到相邻的位置 (也可以视为移动空格到临近的数字的位置)



例3: The 8-Puzzle

7	2	4
5		6
8	3	1

Start State

1	2	3
4	5	6
7	8	

Goal State

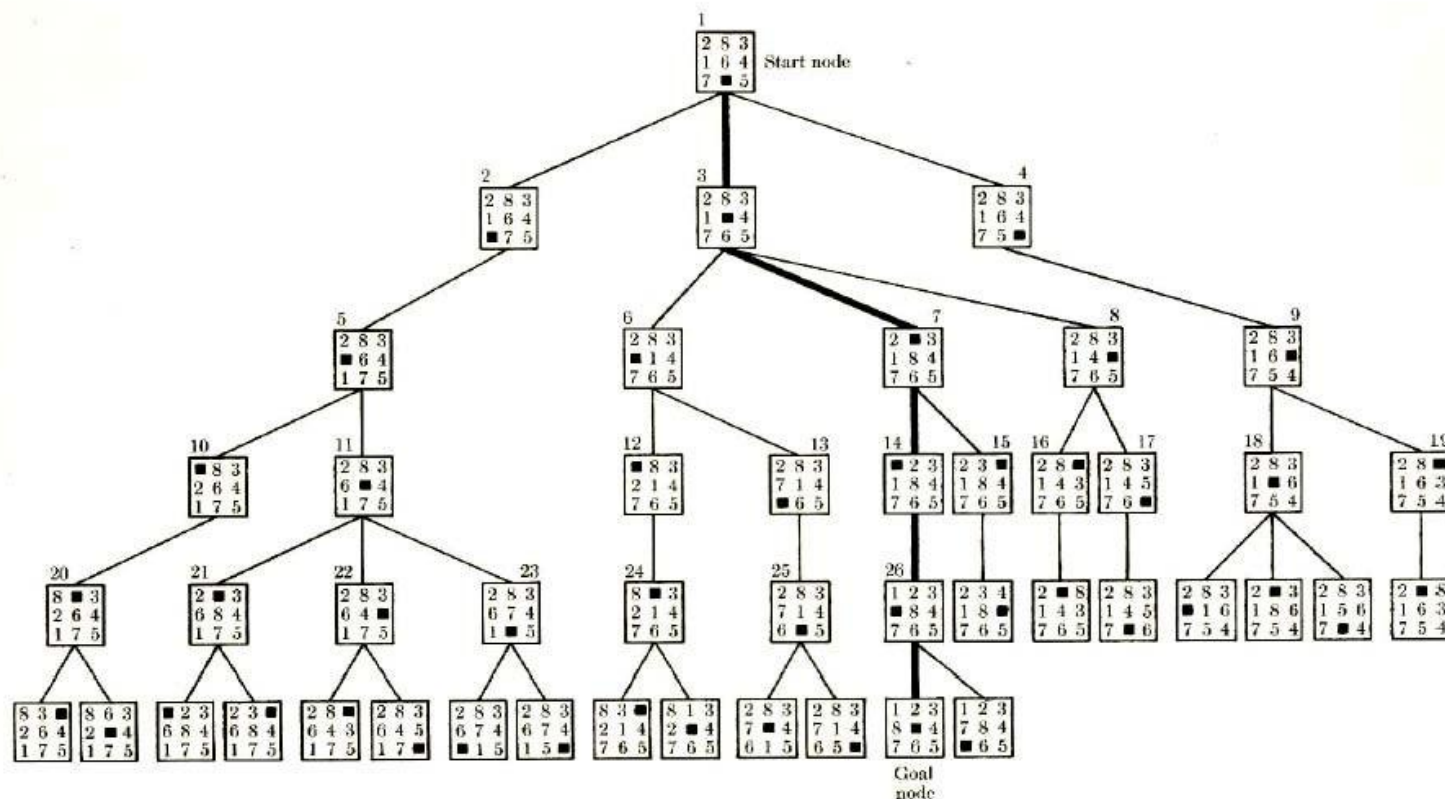
- 状态空间:
各种不同的方格摆放方式
- 初始状态:
左图所示的方格摆放方式
- 目标:
右图所示的方格摆放方式
- 动作:
向上/下/左/右移动空格

规则: 可以把空格移动到相邻的位置

目标: 从初始数字排布变换出目标的数字排布方式



例3: The 8-Puzzle



Search Space for 8-Puzzle Problem



例3: The 8-Puzzle

- 八数码难题属于滑块难题



漫画人物

I	H	E	S
H	P	I	O
P	I	S	R
P	O		H

英语拼词



华容道

- 滑块难题又属于二维组合谜题（2D combination puzzle）
- 此外，还有三维组合谜题（3D combination puzzle），例如魔方



搜索问题形式优化

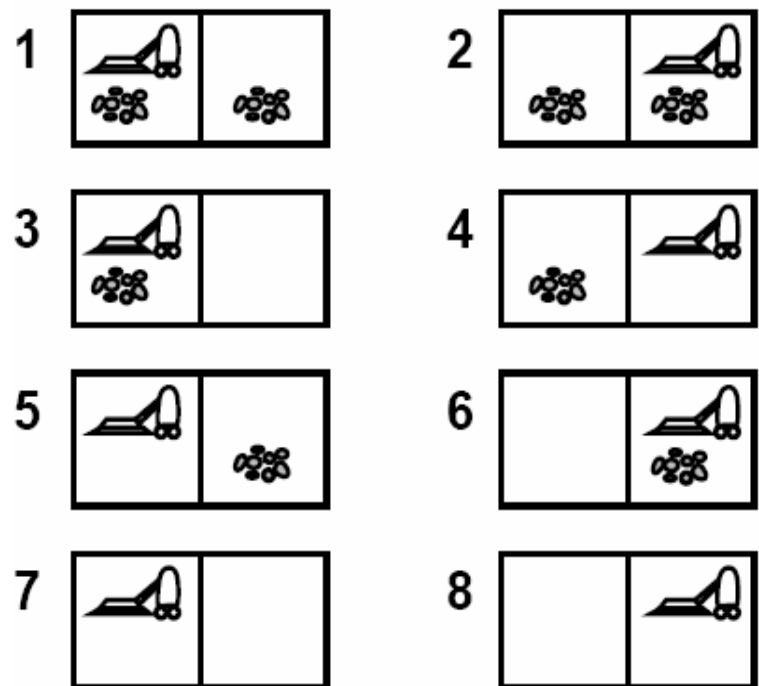
- ◆ 前面的示例中，一个状态是对应一个真实世界中的状态
- ◆ 然而，一个状态也可以对应智能体如何认识世界的情况：智能体的认知状态（the agent's knowledge state.）



例4：吸尘器世界

问题描述：

- 有一个吸尘器需要清扫两个房间
- 每个房间有两个状态：
干净或不干净
- 吸尘器有**向左(left)**和**向右(right)**两个动作（左边/右边没有房间时动作不起作用）
- 吸尘器有**吸尘(suck)**的动作，该动作使得吸尘器所在的房间状态变为**干净**（即使房间本身就是干净的）



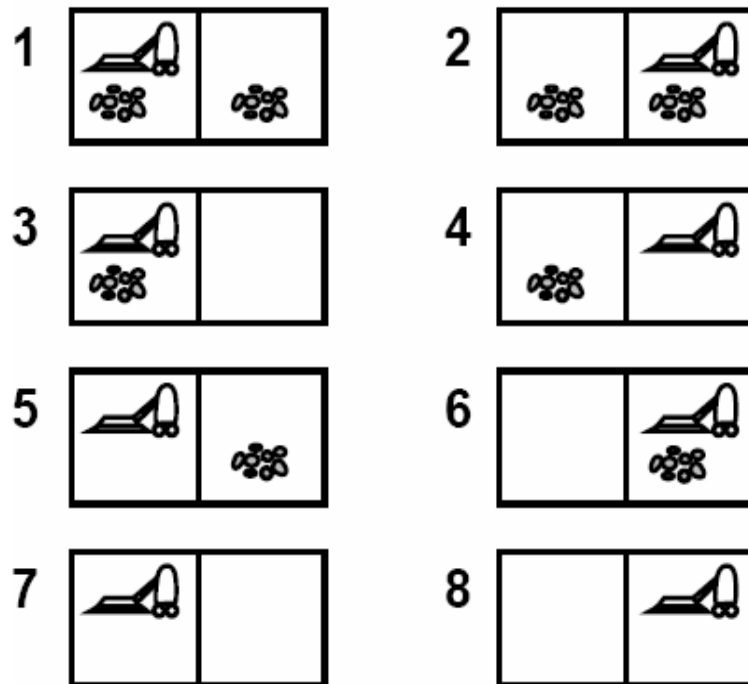
物理状态



例4：吸尘器世界

认知层面的状态空间

- 一个认知状态是物理状态空间的一个子集。也就是智能体知道自己处于几个物理状态中的一个状态中，但是不知道具体是哪一个。



目标是所有房间的状态都为干净



例4：吸尘器世界

认知层面的状态空间

- 完全不清楚物理世界

认知状态是物理状态的集合

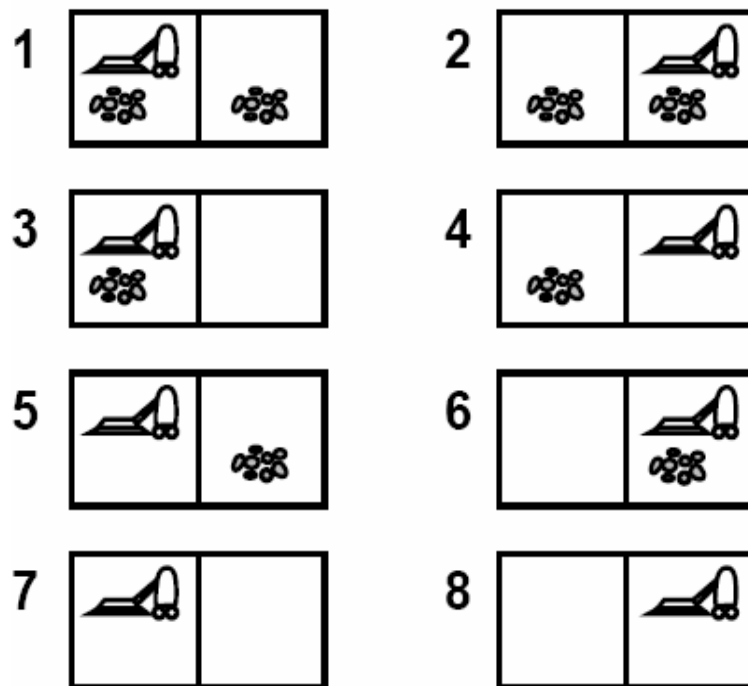
- 初始认知状态为{1,2,3,4,5,6,7,8}

智能体不知道自己身处具体是哪个物理状态

- 但不论如何，动作序列 $\langle \text{right, suck, left, suck} \rangle$ 都可到达目标，认知状态的变化情况如下

$\{1,2,3,4,5,6,7,8\}$ **right** $\rightarrow \{2, 4, 6, 8\}$

suck $\rightarrow \{4, 8\}$ **left** $\rightarrow \{3, 7\}$ **suck** $\rightarrow \{7\}$



目标是所有房间的状态都为干净



搜索算法

算法输入

- 具体的**初始状态**：一个具体的**物理状态**，或是一个物理状态的集合表示的智能体的**认知状态**，等等
- **后继函数**： $S(x) = \{x\text{状态经过一个动作之后可以到达的状态的集合}\}$
- **目标测试**：一个作用于状态上，当该状态满足目标条件时返回真的函数
- **前进成本**： $C(x,a,y) = \text{从}x\text{状态通过动作}a\text{到达}y\text{状态所需要的成本 (}x\text{状态无法到达}y\text{状态时, } \forall_a C(x,a,y) = \infty)$

算法输出

- 从**初始状态**到某个满足**目标测试**的状态的状态序列



获取动作系列

- 状态 x 的后继状态可能来自不同的动作，如：
 - $x \rightarrow a \rightarrow y$
 - $x \rightarrow b \rightarrow z$
- 后继函数 $S(x)$ 返回的状态集合中的状态是状态 x 通过任何一个动作能达到的状态，因此需要把来自不同动作的后继状态加以区分
 - 因此修改 $S(x)$ 的返回值，不仅包括后继状态，还要记录获得这个后继状态所经过的动作
 - $S(x) = \{ \langle y, a \rangle, \langle z, b \rangle \}$
状态 y 通过动作 a 得到，状态 z 通过动作 b 得到
 - $S(x) = \{ \langle y, a \rangle, \langle y, b \rangle \}$
状态 y 通过动作 a 得到，状态 y 通过动作 b 得到



算法框架

- We put states we haven't yet explored or expanded, but want to explore, in a list called the **Frontier边界 (or Open 表)**.
还没有被探索，但准备下一步探索的状态的集合
- Initially, all that is in the Frontier is the **initial state**.
初始边界 = 初始状态集合
- At each iteration, we pull a node from the Frontier, apply **S(x)**, and insert children back into the Frontier.

```
TreeSearch(Frontier, Successors, Goal?)
```

```
  If Frontier is empty return failure
```

```
  Curr = select state from Frontier
```

```
  If (Goal?(Curr)) return Curr.
```

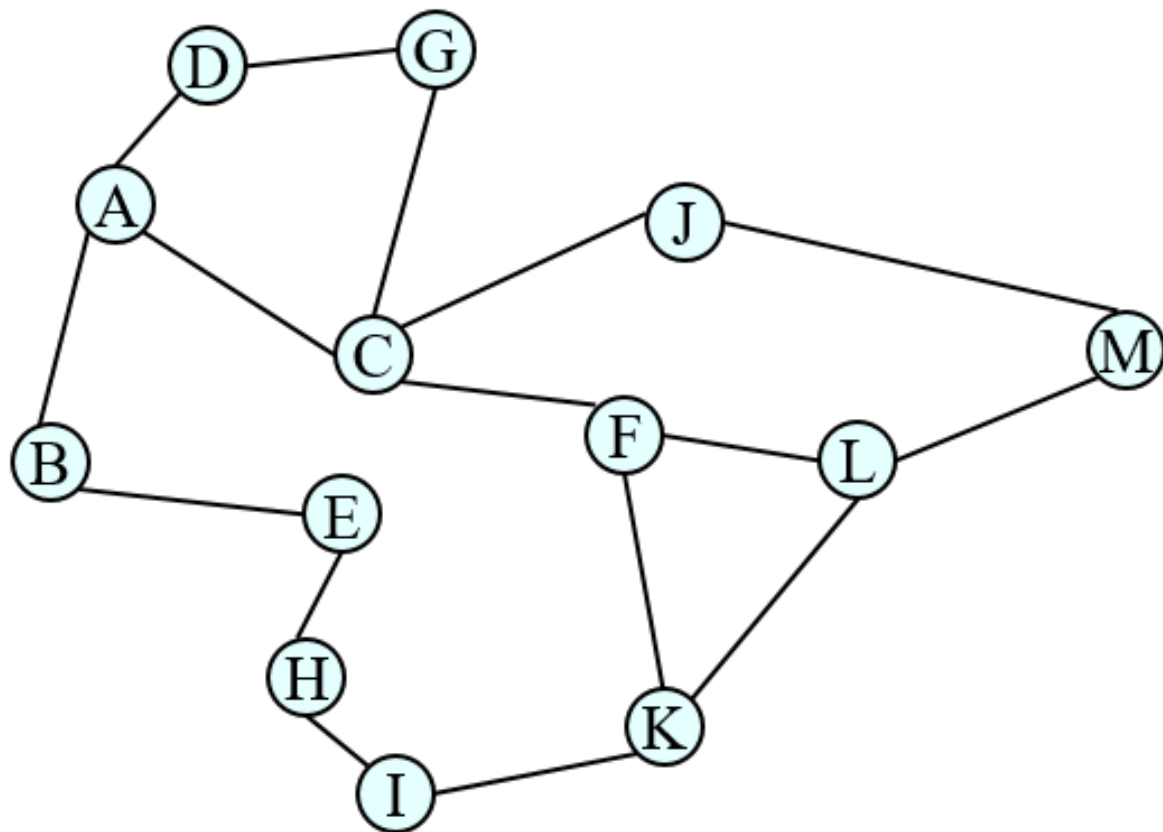
```
  Frontier' = (Frontier - {Curr}) U Successors(Curr)
```

```
  return TreeSearch(Frontier', Successors, Goal?)
```



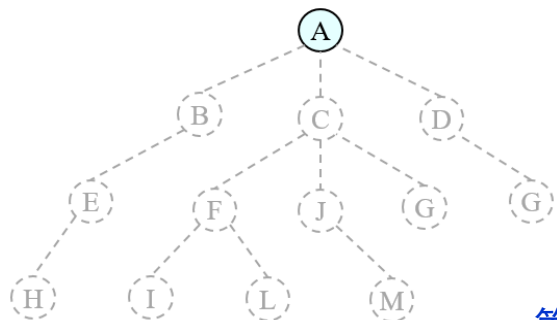
树搜索

- 树搜索：用搜索树来寻找一条从起点 A 到终点 M 的路径。

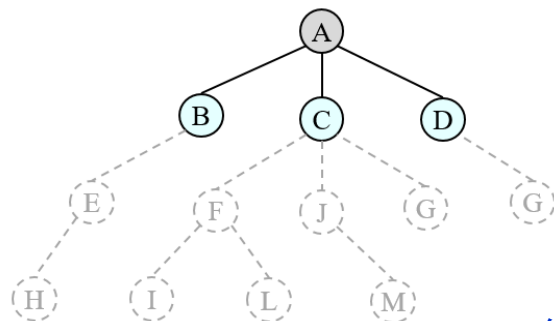




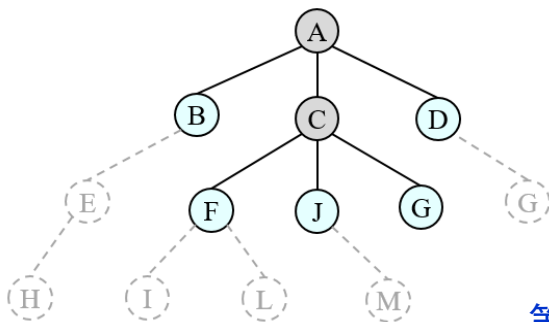
树搜索



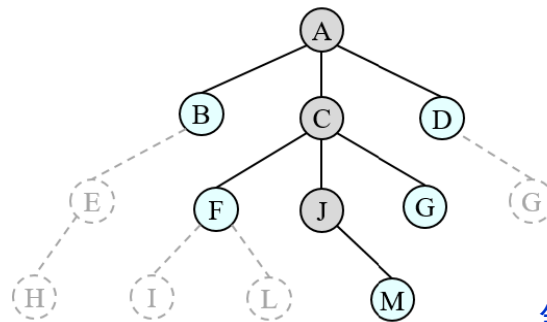
第1步



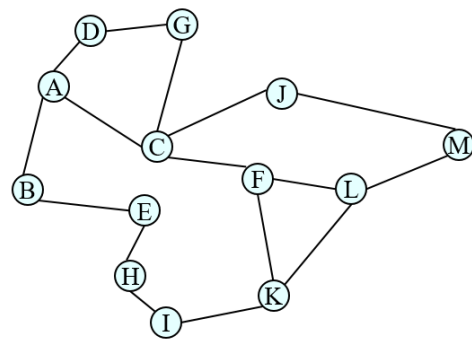
第2步



第3步



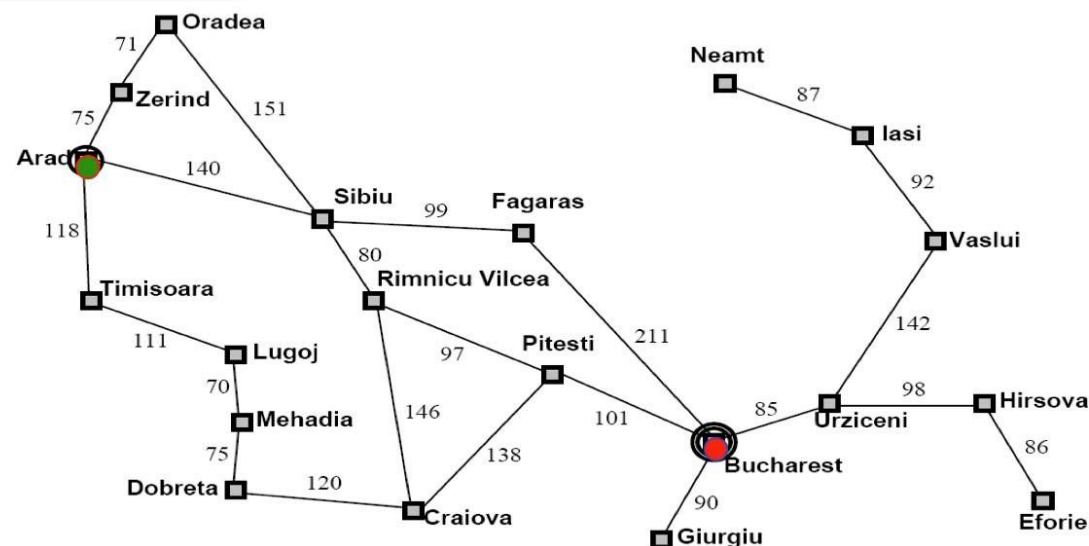
第4步



- ❖ A为根节点、M为叶节点。
- ❖ 粗实线为已生成节点。
- ❖ 虚线表示节点尚未生成。
- ❖ 已扩展节点加阴影表示。



示例：罗马尼亚旅行问题

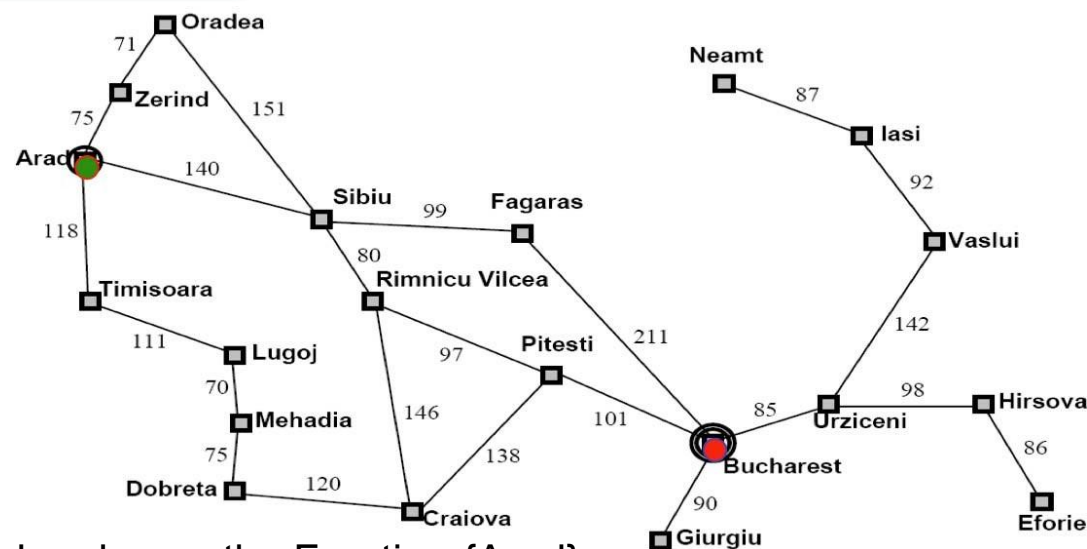


1. Initial nodes on the Frontier: {Arad}.
2. Expand **Arad**: {Z<A>, T<A>, **S<A>**}.
3. Expand **Sibiu**: {Z<A>, T<A>, A<S,A>, O<S,A>, **F<S,A>**, R<S,A>}
4. Expand **Fagaras**: {Z<A>, T<A>, A<S,A>, O<S,A>, R<S,A>, S<F,S,A>, **B<F,S,A>**}

Solution is now on frontier; cost of this solution is **140+99+211 = 450**



示例：罗马尼亚旅行问题

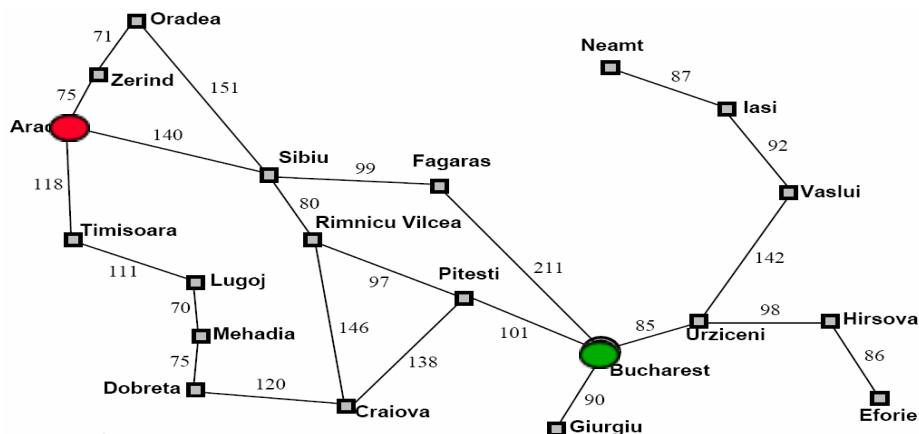


1. Initial nodes on the Frontier: {Arad}.
2. Expand **Arad**: {Z<A>, T<A>, **S<A>**}.
3. Expand **Sibiu**: {Z<A>, T<A>, A<S,A>, O<S,A>, F<S,A>, **R<S,A>**}.
4. Expand **R.V.**: {Z<A>, T<A>, A<S,A>, O<S,A>, R<S,A>, S<R,S,A>, **P<R,S,A>**, C<R,S,A>}.
5. Expand **Pitesti**: {Z<A>, T<A>, A<S,A>, O<S,A>, R<S,A>, S<R,S,A>, P<R,S,A>, C<R,S,A>, R<P,R,S,A>, C<P,R,S,A>, **B<P,R,S,A>**}.

Solution is now on frontier; cost of this solution is **140+80+97+101= 418**



示例：罗马尼亚旅行问题



Solution 2:

{Arad}

{Z<A>, T<A>, S<A>},

{Z<A>, T<A>, A<S,A>, O<S,A>, F<S,A>, R<S,A>}

{Z<A>, T<A>, A<S,A>, O<S,A>, F<S,A>, S<R,S,A>, P<R,S,A>, C<R,S,A>}

{Z<A>, T<A>, A<S,A>, O<S,A>, F<S,A>, S<R,S,A>, C<R,S,A>, R<P,R,S,A>, C<P,R,S,A>, Bucharest<P,R,S,A>}

Solution: Arad -> Sibiu -> Rimnicu Vilcea -> Pitesti -> Bucharest **Cost:**

140 + 80 + 97 + 101 = **418**

Solution 1:

{Arad},

{Z<A>, T<A>, S<A>},

{Z<A>, T<A>, A<S;A>, O<S;A>, F<S;A>, R<S;A>}

{Z<A>, T<A>, A<S;A>, O<S;A>, R<S;A>, S<F;S;A>, B<F;S;A>}

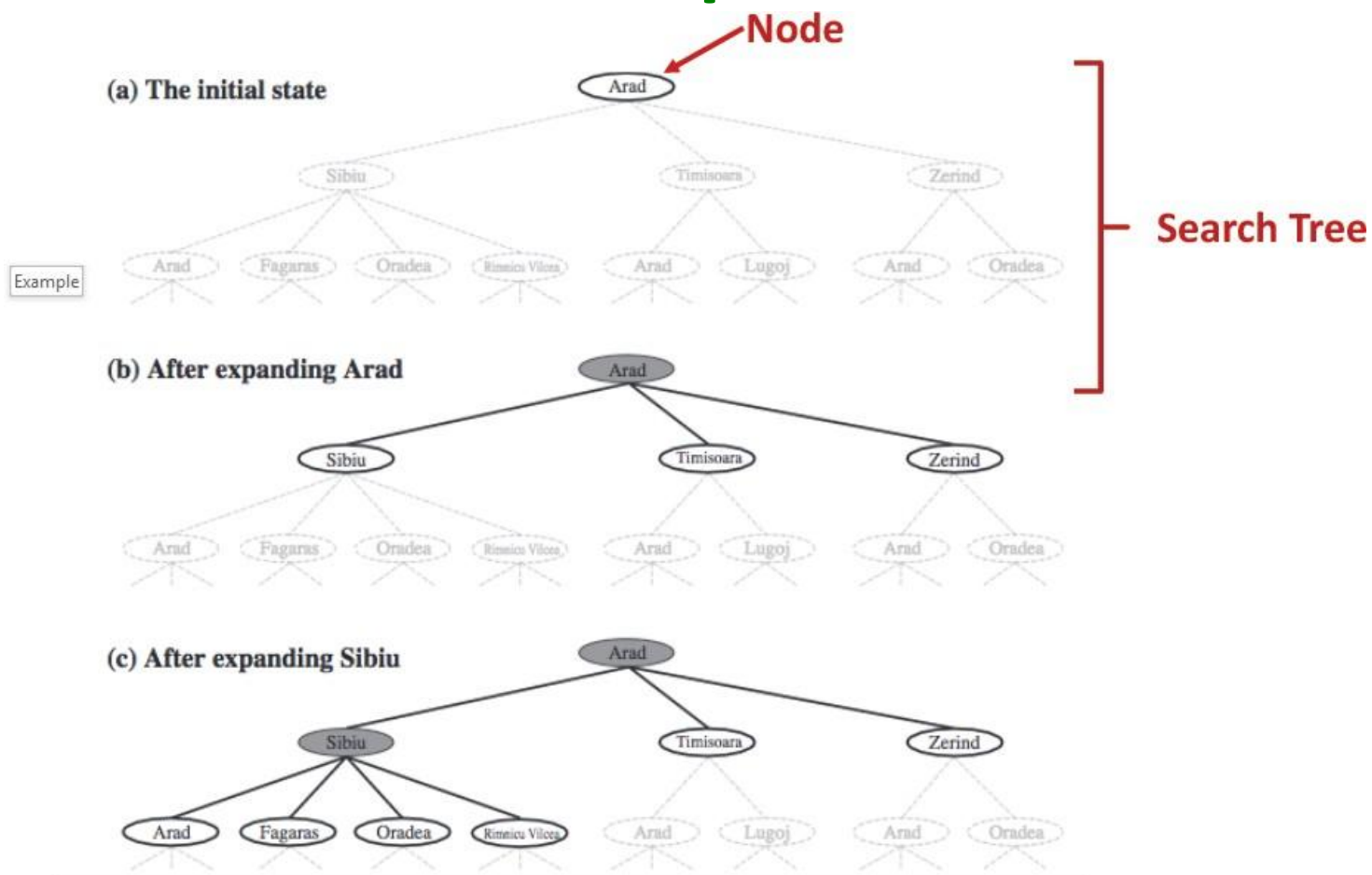
Solution: Arad -> Sibiu -> Fagaras -> Bucharest

Cost: 140 + 99 + 211 = 450



示例：罗马尼亚旅行问题

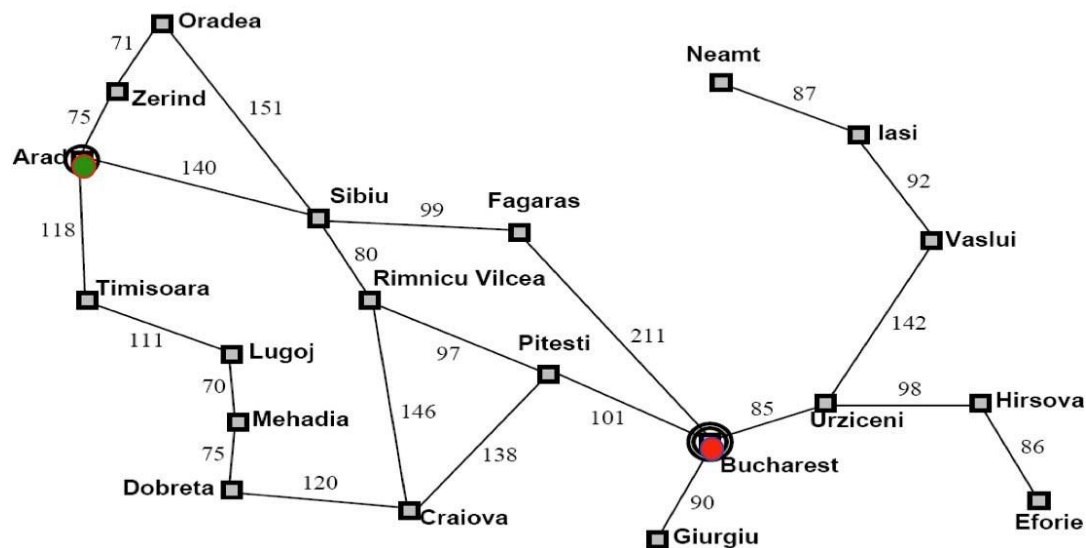
Search Tree Representation





示例：罗马尼亚旅行问题

Reflections on Example

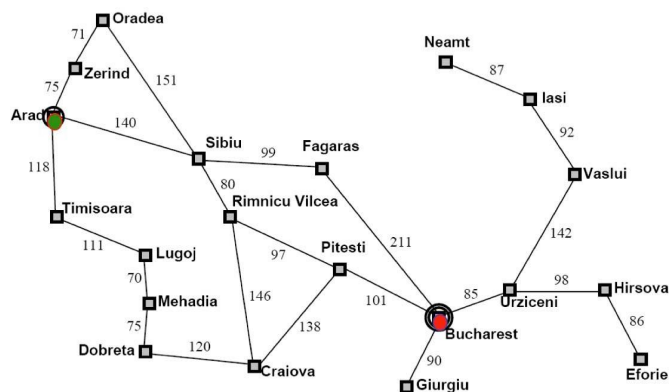


In this problem, the Frontier here contains a set of **paths**, not just **states**. 边界不仅是状态的集合而是路径的集合



示例：罗马尼亚旅行问题

Reflections on Example

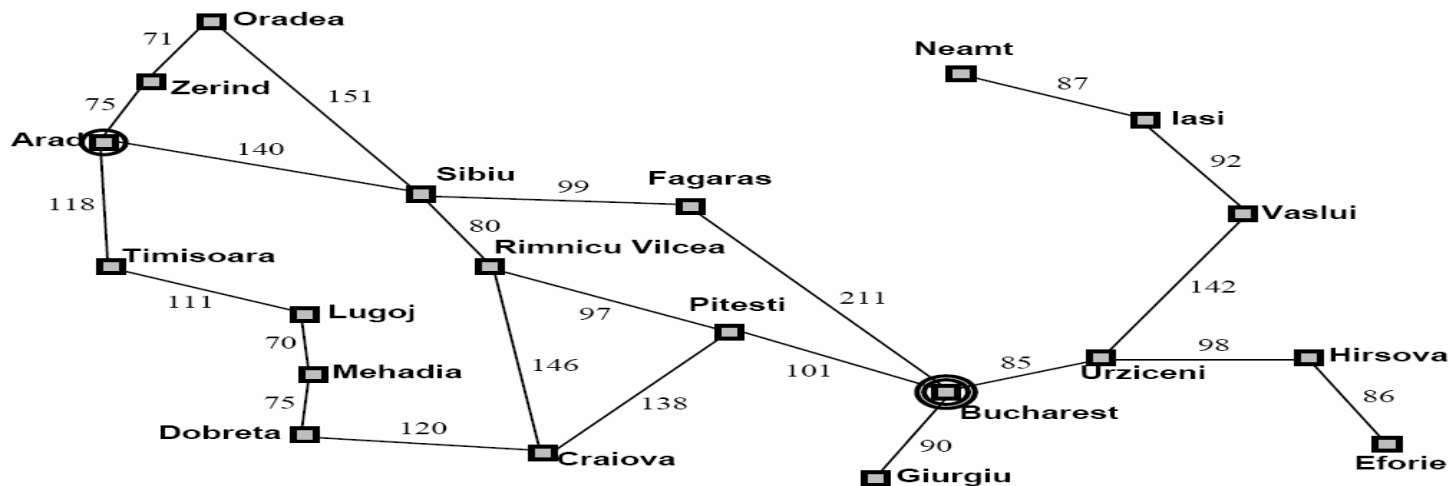


The **order** states are selected from the Frontier has **a critical effect** on
表明了**状态选择的顺序**会对搜索操作**产生重要的影响**

- Whether or not a solution is found会影响搜索是否能找到解
- The **cost** of the solution that is found.会影响搜索到的解的成本大小
- The **time** and **space** required by the search.会影响搜索过程中所需的时间和空间资源



循环问题



{Arad}

{Z<A>, T<A>, S<A>},

{Z<A>, T<A>, O<S;A>, F<S;A>, A<S;A>, R<S;A>}

{Z<A>, T<A>, O<S;A>, F<S;A>, R<S;A>, Z<A;S;A>, T<A;S;A>, S<A,S;A>}

边界不是状态的集合而是路径的集合，因此只要路径不同就会往边界上添加新的元素，导致了循环问题



搜索算法的重要特征

- **完备性** (Completeness) : 搜索算法是否总能在问题存在解的情况下找到解
- **最优性** (Optimality) : 当问题中的动作是需要成本时, 搜索算法是否总能找到成本最小的解
- **时间复杂度** (Time complexity) : 搜索算法最多需要探索/生成多少个节点来找到解
- **空间复杂度** (Space complexity) : 搜索算法最多需要将多少个节点储存在内存中

盲目搜索



选择法则

上面的例子表明了状态选择的顺序会对搜索操作产生重要的影响:

- 会影响搜索是否能找到解
- 会影响搜索到的解的成本大小
- 会影响搜索过程中所需要的时间和空间资源

搜索算法的重要性质

- **完备性 (Completeness)** : 搜索算法是否总能在问题存在解的情况下找到解
- **最优性 (Optimality)** : 当问题中的动作是需要成本时, 搜索算法是否总能找到成本最小的解
- **时间复杂度 (Time complexity)** : 搜索算法最多需要探索/生成多少个节点来找到解
- **空间复杂度 (Space complexity)** : 搜索算法最多需要将多少个节点储存在内存中



盲目搜索策略

- 这些策略都采用固定的规则来选择下一需要被扩展的状态
- 这些规则不会随着要搜索解决的问题的变化而变化
- 这些策略不考虑任何与要解决的问题领域相关的信息

常用的盲目搜索方法

- 宽度优先 (Breadth-First)
- 深度优先 (Depth-First)
- 一致代价 (Uniform-Cost)
- 深度受限 (Depth-Limited)
- 迭代加深搜索 (Iterative-Deepening search)



边界中的节点选择

- Selection can be achieved by employing an appropriate ordering of the frontier set, i.e.:
 1. Order the elements on the Frontier.
对边界上的元素进行排序
 2. Always select the first element.
总是选择第一个元素
- Any selection rule can be achieved by employing an appropriate ordering of the frontier set.
任何选择规则都可以视为对边界采用某种合适的排序方式



宽度优先搜索

- 把当前要扩展的状态的后继状态放在边界的**最后** $\{0<>\}$
 - 例子:
 - 假设使用正整数表示状态 $\{0,1,2,\dots\}$
 - 状态 n 的后继状态为状态 $n+1$ 和状态 $n+2$
 - 如: $S(1) = \{2, 3\}$; $S(10) = \{11, 12\}$
 - 初始状态为 0
 - 目标状态为 5
- $\{1,2\}$
- $\{2,2,3\}$
- $\{2,3,3,4\}$
- $\{3,3,4,3,4\}$
- $\{3,4,3,4,4,5\}$
- ...



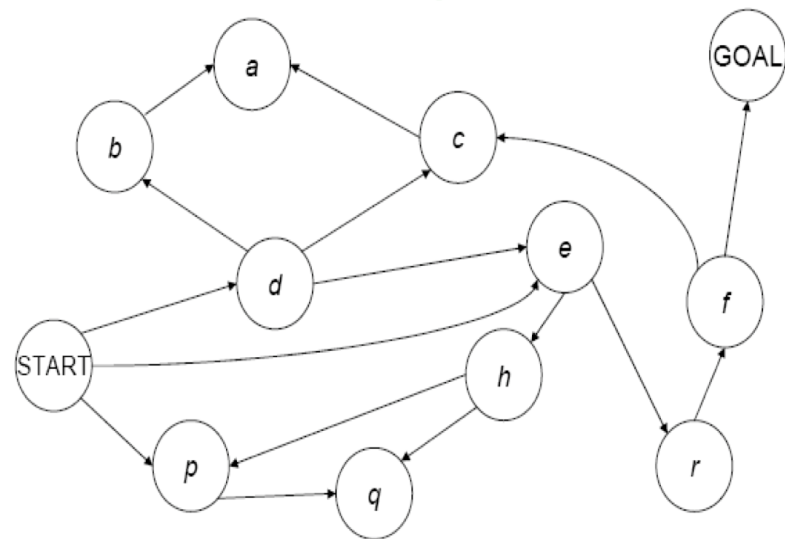
宽度优先搜索

- 把当前要扩展的状态的后继状态放在边界的**最后** $\{0<>\}$
 - 例子:
 - 假设使用正整数表示状态 $\{0,1,2,\dots\}$
 - 状态 n 的后继状态为状态 $n+1$ 和状态 $n+2$
 - 如: $S(1) = \{2, 3\}$; $S(10) = \{11, 12\}$
 - 初始状态为 0
 - 目标状态为 5
- $\{1,2\}$
- $\{2,2,3\}$
- $\{2,3,3,4\}$
- $\{3,3,4,3,4\}$
- $\{3,4,3,4,4,5\}$
- ...



宽度优先搜索

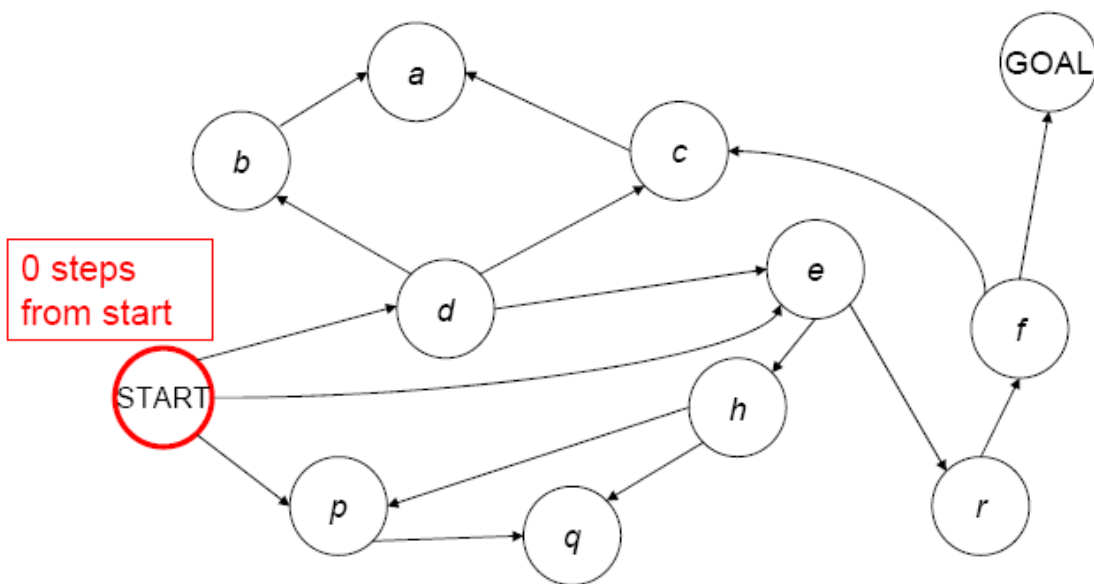
1. Place Start in the Frontier.
2. Expand all nodes reachable from Start in 1 step, but not more than 1; add path to back of Frontier list.
3. Expand all nodes reachable from Start in 2 step, but not more than 2; add path to back of Frontier list.
4. Expand all nodes reachable from Start in 3 step, but not more than 3; add to path back of Frontier list.
5. And so on





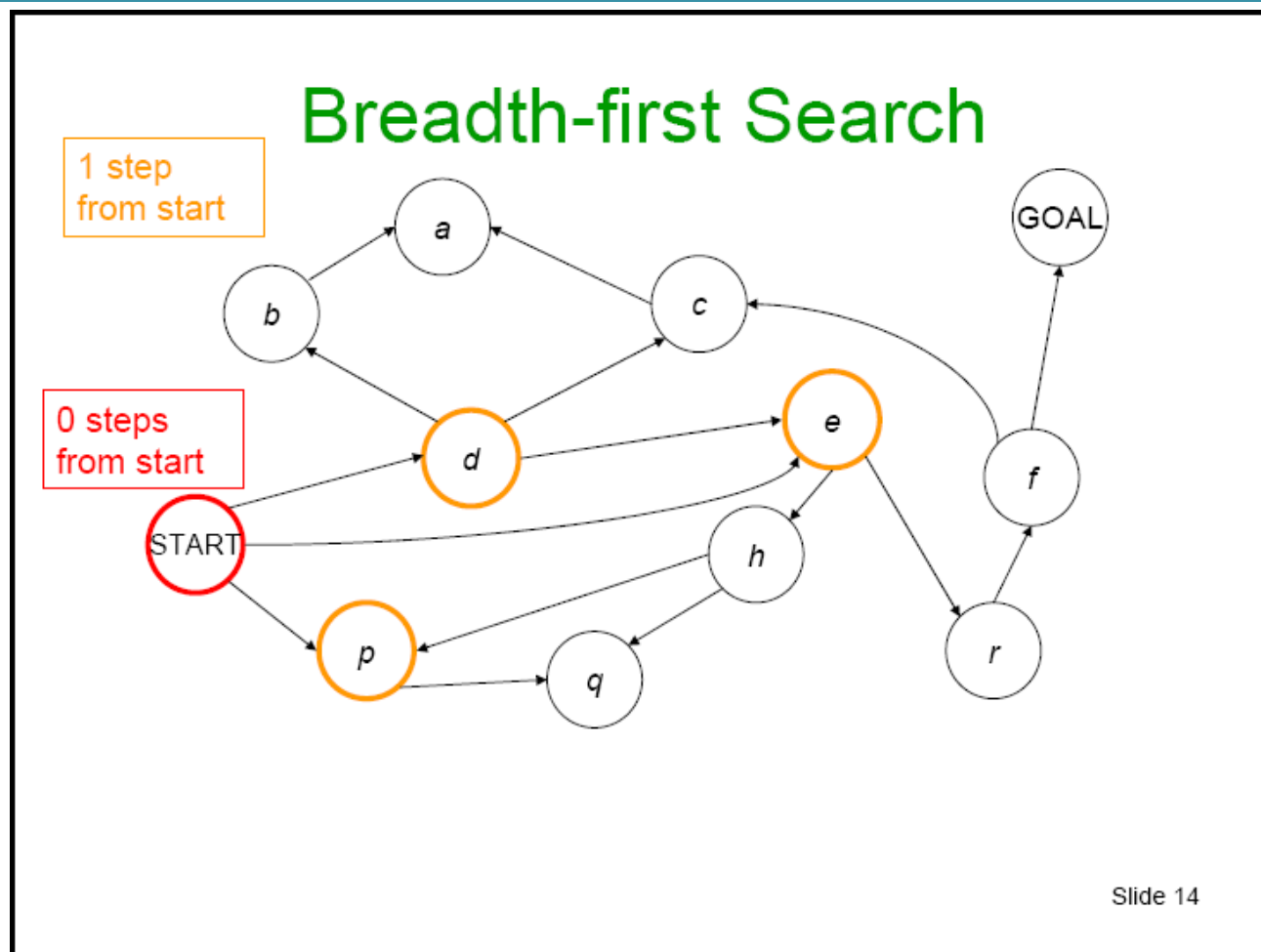
宽度优先搜索

Breadth-first Search

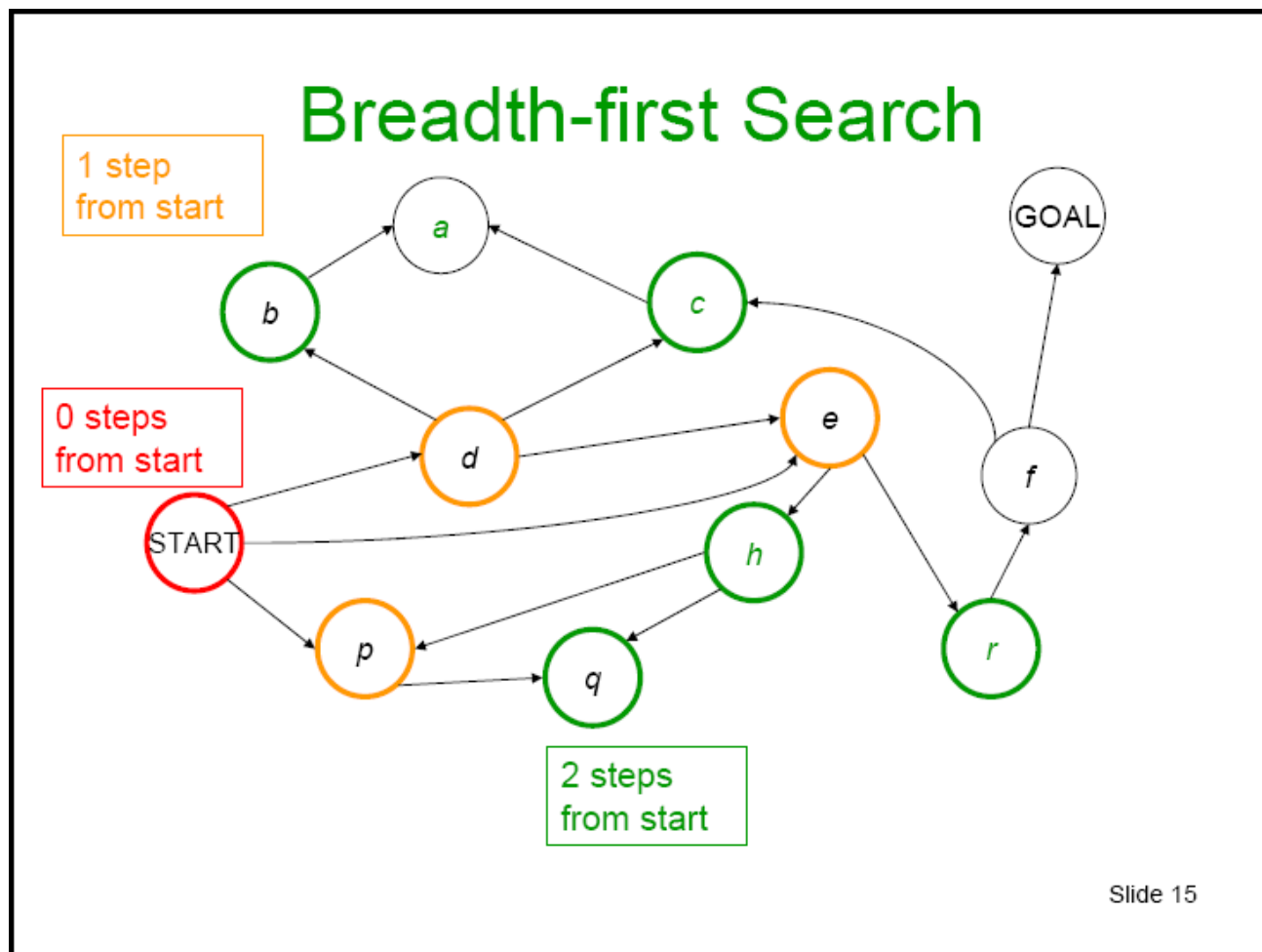


Slide 13

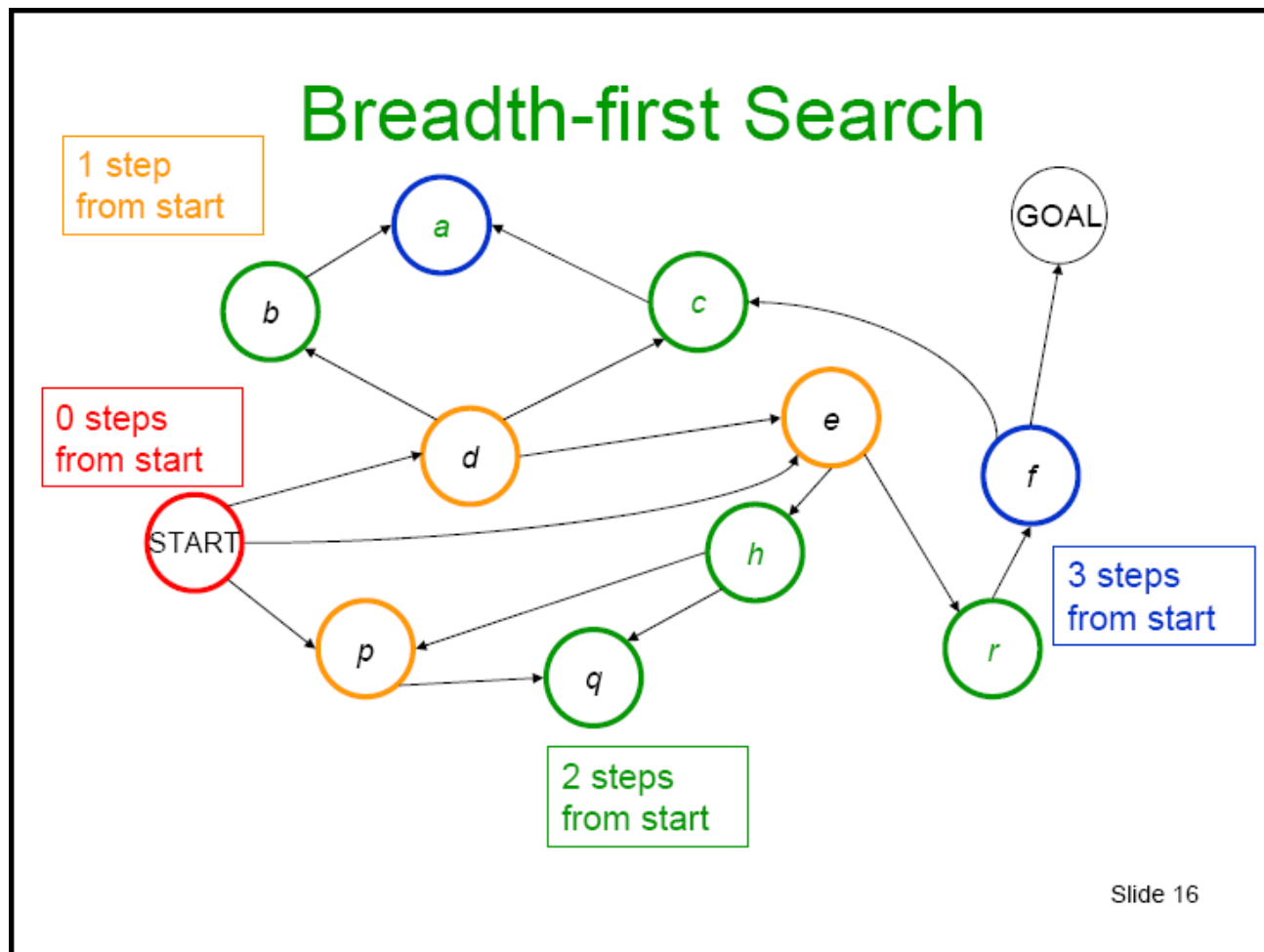
宽度优先搜索



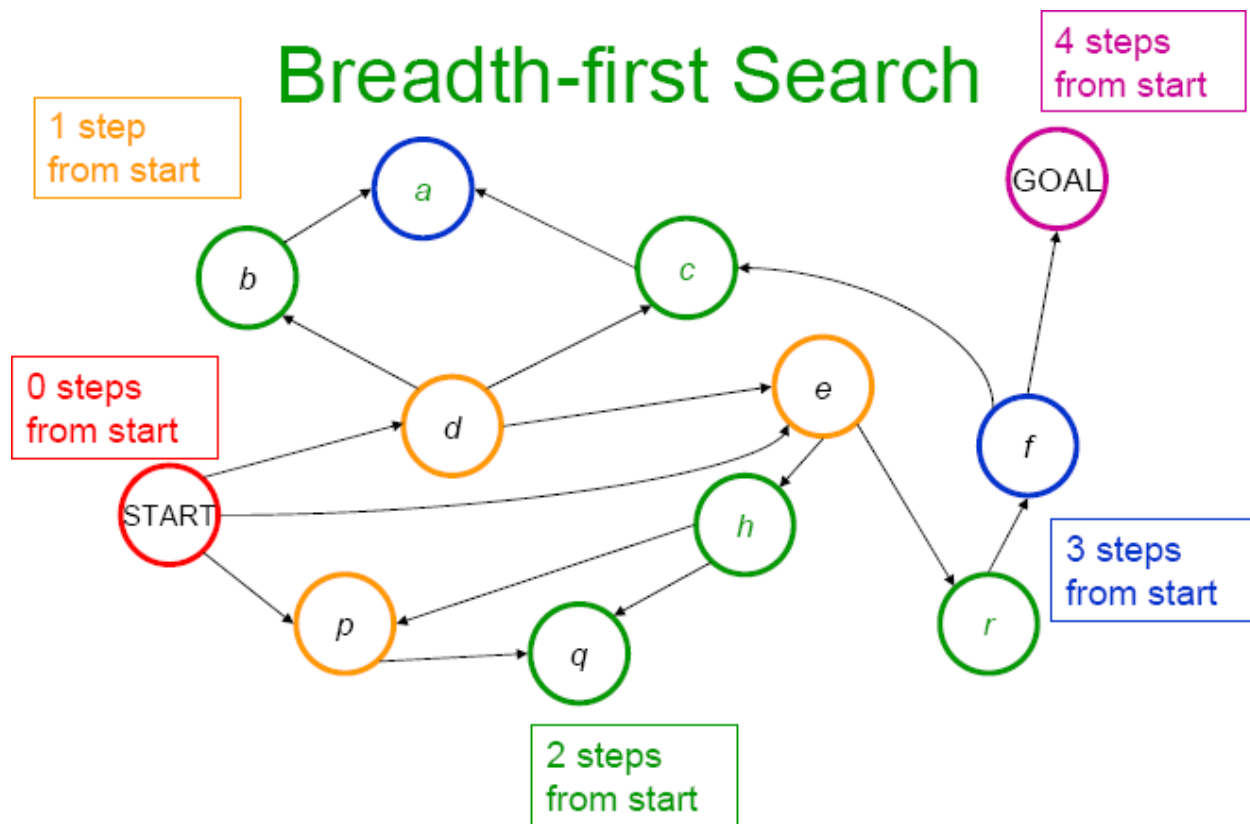
宽度优先搜索



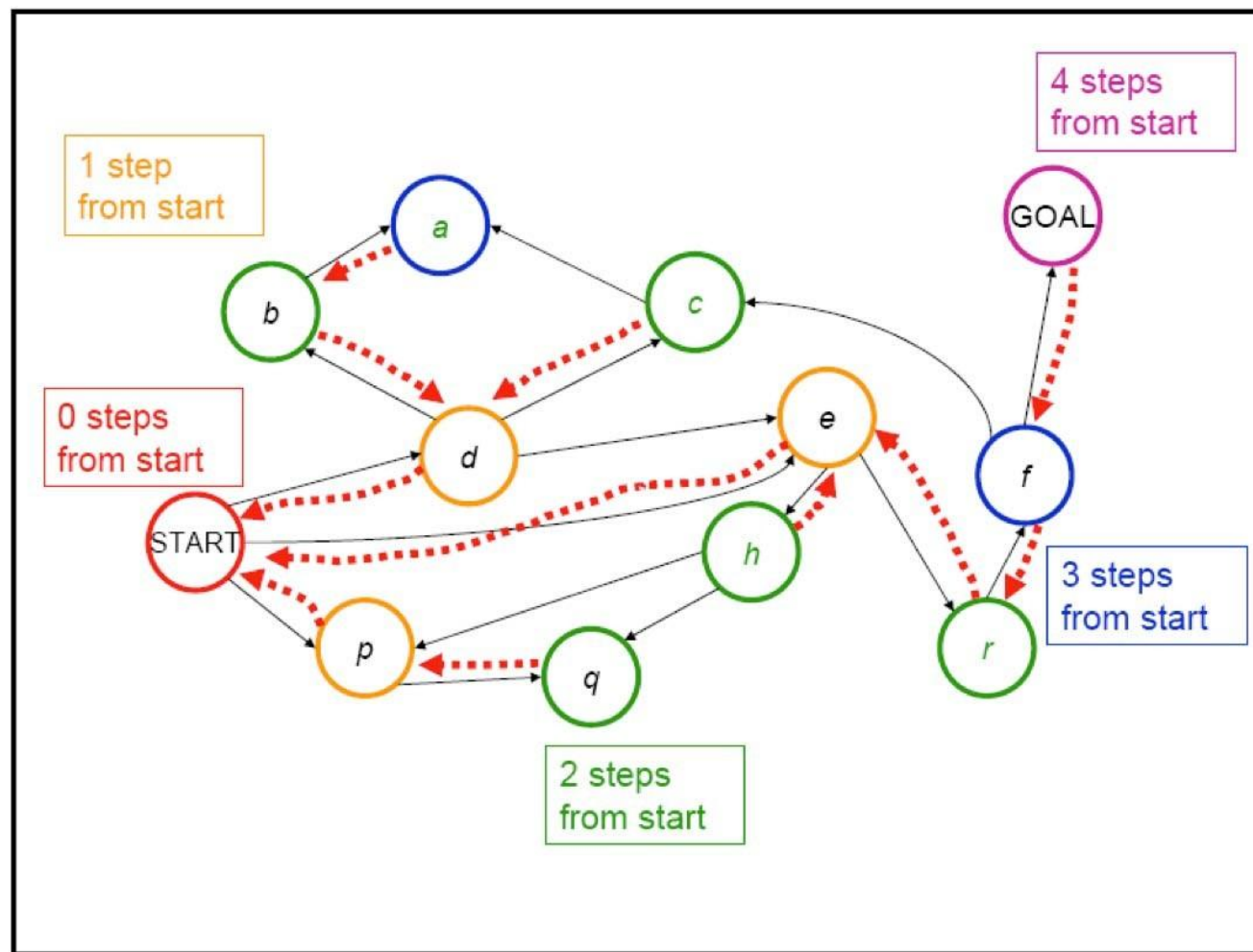
宽度优先搜索



宽度优先搜索

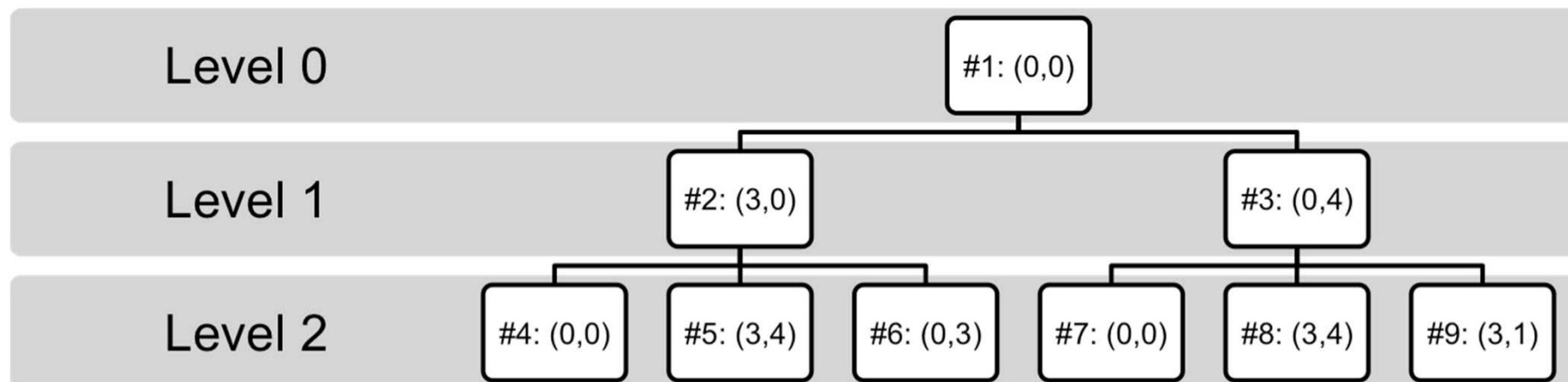


宽度优先搜索





宽度优先搜索 for Water Jug



In the tree above we order the states explored; paths to states are represented by the path from the root to that states.

Breadth-First Search explores the search space level by level.



宽度优先搜索 for Water Jug

initial state = (0,0), goal state = (*,2), actions
(successor functions): Empty-3-Gallon, Empty-4-
Gallon, Fill-3-Gallon, Fill-4- Gallon, Pour-3-into-4,
Pour 4-into-3.

1. Frontier = {<(0,0)>}

2. Frontier = {<(0,0),(3,0)>, <(0,0),(0,4)>}

3. Frontier = {<(0,0),(0,4)>, <(0,0),(3,0),(0,0)>,
 <(0,0),(3,0),(3,4)>, <(0,0),(3,0),(0,3)>}

4. Frontier = {<(0,0),(3,0),(0,0)>, <(0,0),(3,0),(3,4)>,
 <(0,0),(3,0),(0,3)>, <(0,0),(0,4),(0,0)>,
 <(0,0),(0,4),(3,4)>, <(0,0),(0,4),(3,1)>}

宽度优先的性质

b = 问题中一个状态最大的后继状态个数

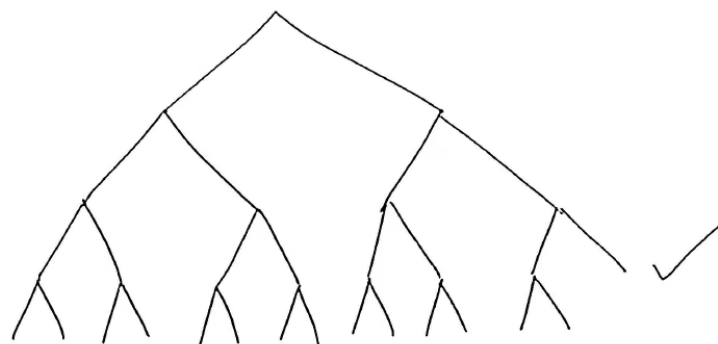
d = 最短解的动作个数

宽度优先搜索具有**完备性**和**最优性**

- 短的路径会在任何比它长的路径之前被遍历
- 给定路径长度，该长度的路径是有限的
- 最终可以遍历所有长度为 d 的路径，因此一定可以找出最短的解

■ **时间复杂度**: $1 + b + b^2 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$

■ **空间复杂度**: $b(b^d - 1) = O(b^{d+1})$





空间复杂度会带来的问题

- 假设 $b = 10$, 并且每秒扩展1000个节点, 每个节点需要100bytes来存储:

Depth	Nodes	Time	Memory
1	1	1 millisec.	100 bytes
6	10^6	18 mins.	111 MB
8	10^8	31 hrs.	11 GB

- 实际情况下, 内存需求会先于时间限制算法的运行



深度优先搜索

- 把当前要扩展的状态的后继状态放在边界的最前面
- 边界上总是扩展最深的那个节点

与宽度优先搜索的示例比较：

深度优先：

{0}

{1,2}

{2,3,2}

{3,4,3,2}

{4,5,4,3,2}

{5,6,4,5,4,3,2}

...

宽度优先：

{0}

{1,2}

{2,2,3}

{2,3,3,4}

{3,3,4,3,4}

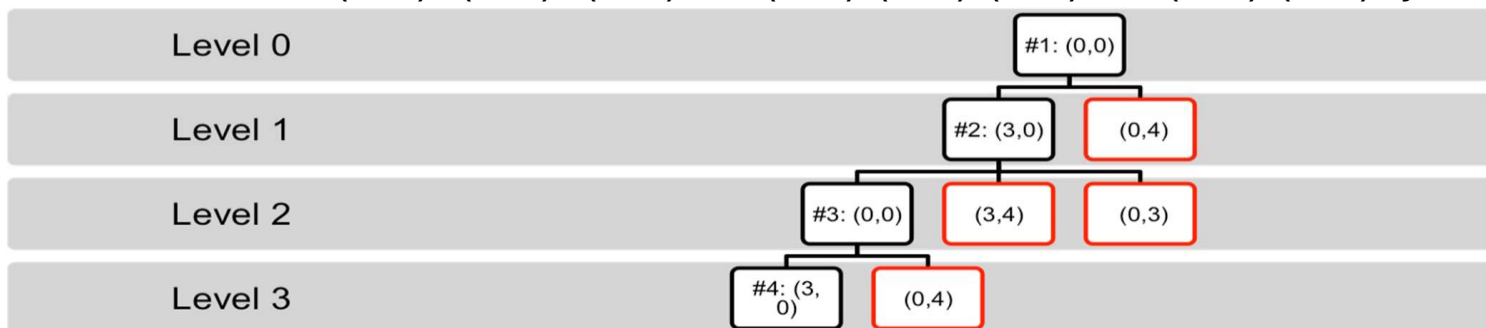
{3,4,3,4,4,5}

...

深度优先搜索

initial state = (0,0), goal state = (*,2), actions (successor functions)
= Empty-3-Gallon, Empty-4-Gallon, Fill-3-Gallon, Fill-4-Gallon, Pour-3- into-4,
Pour 4-into-3.

1. Frontier = {<(0,0)>}
2. Frontier = {<(0,0), (3,0)>, <(0,0), (0,4)>}
3. Frontier = {<(0,0),(3,0),(0,0)>, <(0,0),(3,0),(3,4)>, <(0,0),(3,0),(0,3)>, <(0,4),(0,0)>}
4. Frontier = {<(0,0),(3,0),(0,0),(3,0)>, <(0,0),(3,0),(0,0),(0,4)> <(0,0), (3,0), (3,4)>, <(0,0),(3,0),(0,3)>, <(0,0),(0,4)>}



Red nodes are backtrack points (these nodes remain on Frontier).



深度优先的性质

完备性:

- 在状态空间无限的情况下: No
- 在状态空间有限, 但是存在无限的路径 (例如存在回路) 的情况下: No
 $e.g., S(0) = \{1, 2\}, S(1) = \{0\}, \text{init state } 0, \text{goal is } 2$
- 在状态空间有限, 且对重复路径进行剪枝的情况下: Yes

最优性: No 存在多条路径时, 最先发现的不一定最优



时间复杂度

时间复杂度为： $O(b^m)$

其中 m 是遍历过程中最长路径的长度 (Could explore each branch of search tree)

当 m 远远大于 d 时，时间效率会很差

当存在多条解路径的情况下深度优先搜索可以比宽度优先搜索更快找到解 (可以碰运气先遍历了到达解的那条路径).

空间复杂度

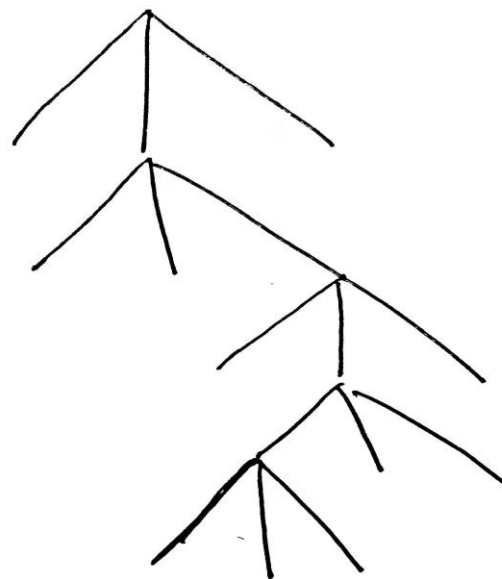
空间复杂度:

深度优先回溯点 = 当前路径上的点的未扩展过的兄弟节点

一次只会考虑一条路径

边界上只包含当前探索的最深的节点, 以及回溯点

$O(bm)$, 线性复杂度是深度优先搜索一个显著的优点





一致代价搜索

- 边界中，按路径的成本升序排列
- 总是扩展成本最低的那条路径
- 当每种动作的成本是一样的时候，和宽度优先是一样的



一致代价搜索性质

- 假设每个动作的成本 $\geq s > 0$
- 一致代价搜索中，所有成本较低的路径都会在成本高的路径之前被扩展
- 给定成本，该成本的路径数量是有限的
- 成本小于最优路径的路径数量是有限的
- 最终，我们可以找到最短的路径

当最优解的路径长度为 d 时，宽度优先搜索的时间和空间复杂度都是 $O(b^{d+1})$

对于一致代价搜索，当最优解的成本为 C^* ，则时间和空间复杂度为 $O(b^{C^*/s+1})$



深度受限搜索

- 宽度优先搜索存在空间复杂度过大的问题
- 深度优先搜索存在可能运行时间非常长，甚至在存在无限路径时无限运行下去的问题
- 深度受限搜索
 - 深度优先搜索，但是预先限制了搜索的深度 L
 - 因此无限长度的路径不会导致深度优先搜索无法停止的问题
 - 但只有当解路径的长度 $\leq L$ 时，才能找到解



深度受限搜索

DLS (Frontier, Successors, Goal?) /* Call with Frontier = {<START>} */

```
WHILE (Frontier not EMPTY) {  
    n= select first node from Frontier  
    Curr = terminal state of n  
    If(Goal?(Curr)) return n
```

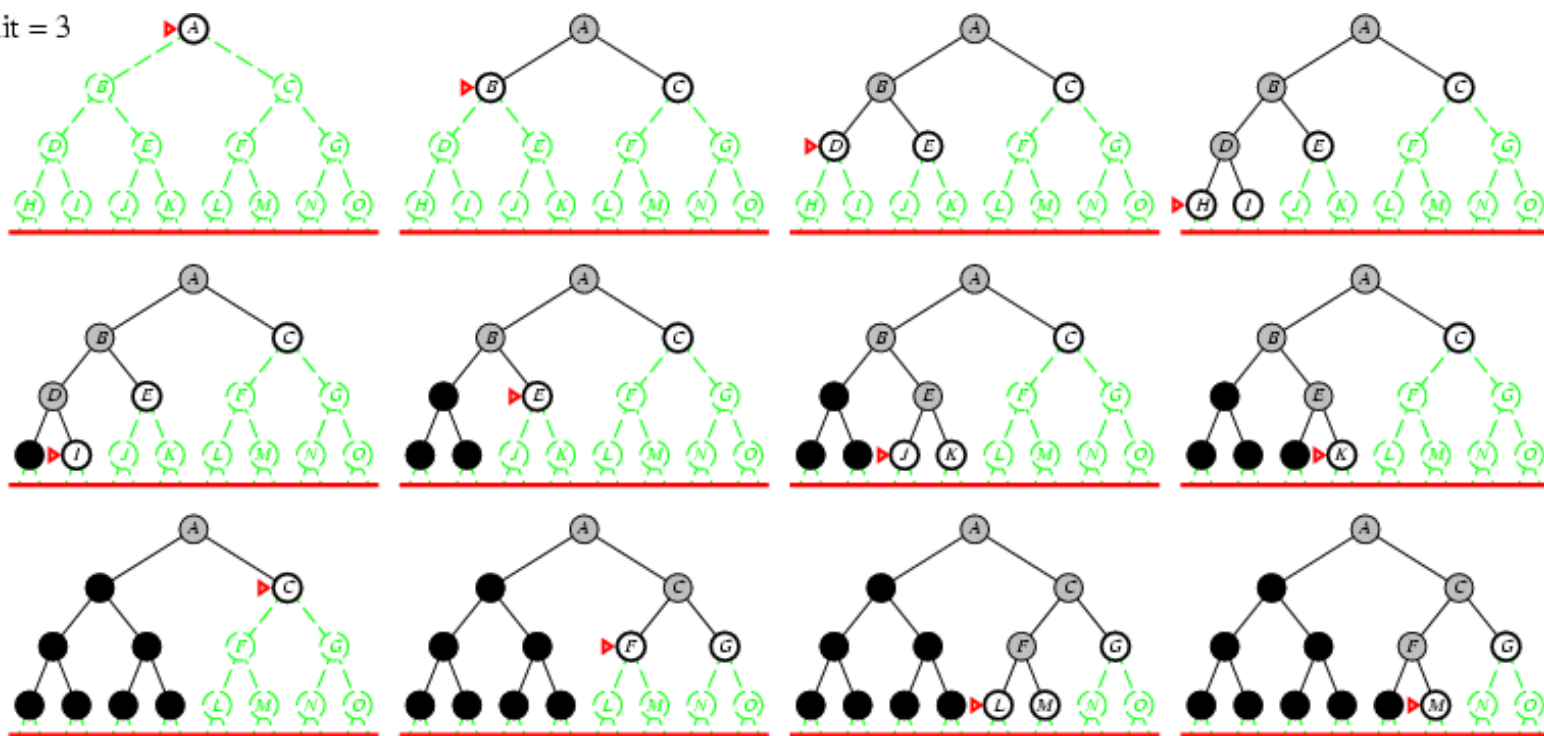
```
    If Depth(n) < D //Don't add successors if Depth(n) = D!!  
        Frontier= (Frontier- {n}) U Successors(Curr)
```

```
    Else  
        Frontier= Frontier- {n}  
        CutOffOccured = TRUE.
```

```
}  
return FAIL
```

示例

Limit = 3





深度受限的性质

- 完备性: No
- 最优性: No
- 时间复杂度: $O(b^L)$
- 空间复杂度: $O(bL)$

L 为限制的最大深度



迭代加深搜索

- 为了解决深度优先搜索和宽度优先搜索存在的问题
- 一开始设置深度限制为 $L = 0$ ，我们迭代地增加深度限制，对于每个深度限制都进行深度受限搜索
- 如果找到解，或者深度受限搜索没有节点可以扩展的时候可以停止当前迭代，并提高深度限制 L
- 如果没有节点可以被剪掉（深度限制不能再提高）仍然没有找到解，那么说明已经搜索所有路径，因此这个搜索不存在解



示例

Limit = 0



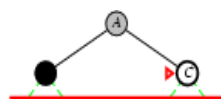


示例

Limit = 0



Limit = 1



示例

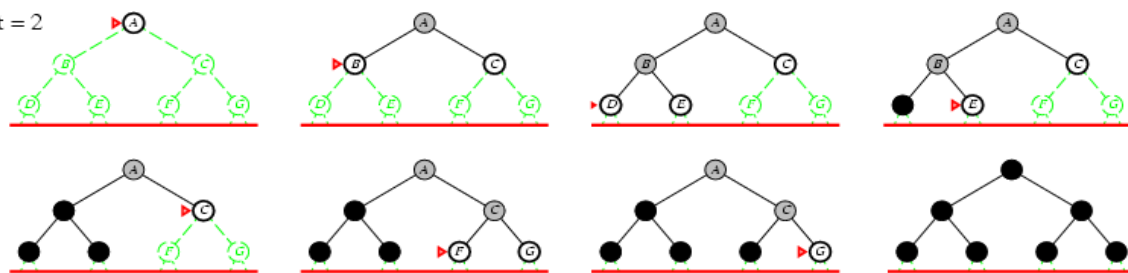
Limit = 0



Limit = 1



Limit = 2



示例

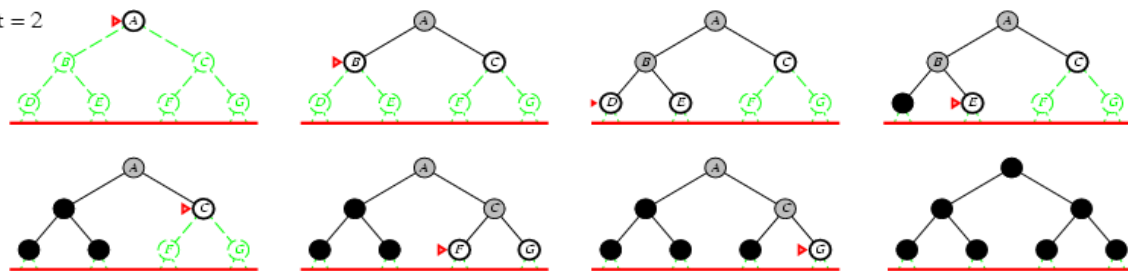
Limit = 0



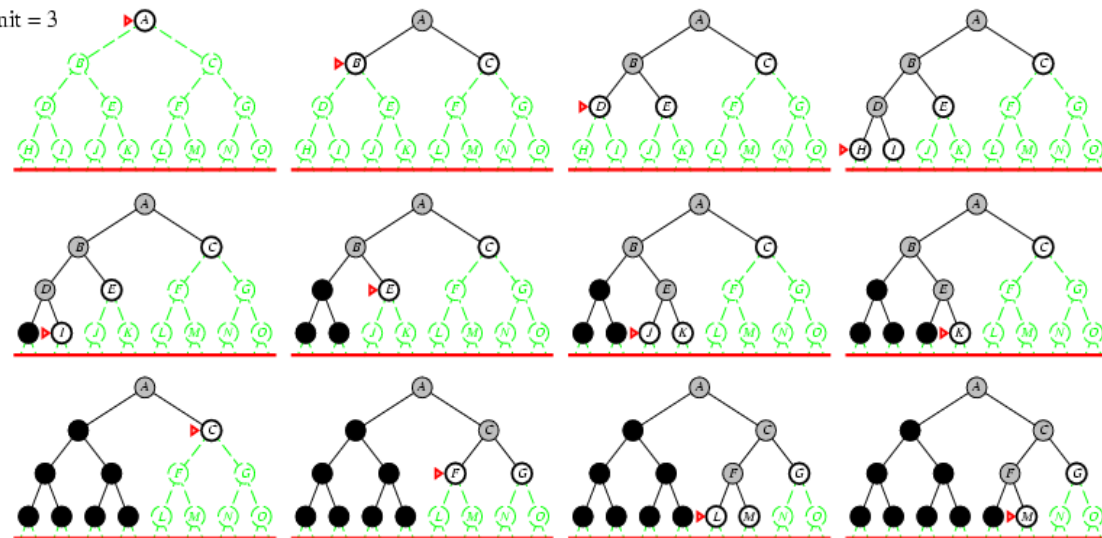
Limit = 1



Limit = 2



Limit = 3





迭代加深搜索性质

完备性: Yes

最优性: Yes (在每个动作的成本一致的情况下)

如果动作成本不一致, 则可以使用成本边界(cost bound)代替深度限制 L :

- 只扩展成本低于成本边界(cost bound)的路径
- 每次迭代时记录当前还未扩展路径中的最小成本
- 下一次迭代则提高成本边界

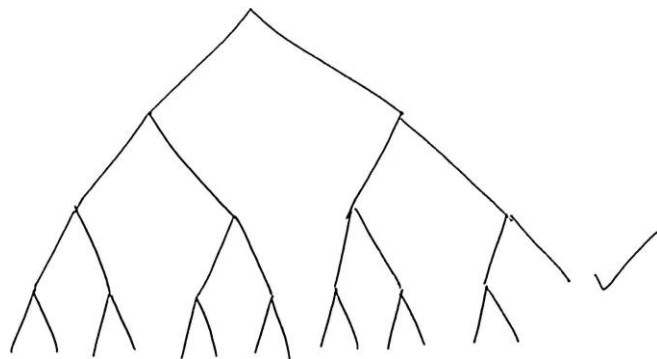
这样开销会很大, 迭代数量为成本数值的构成的集合的大小

时间复杂度: $(d + 1)b^0 + db + (d - 1)b^2 + \dots + b^d = O(b^d)$

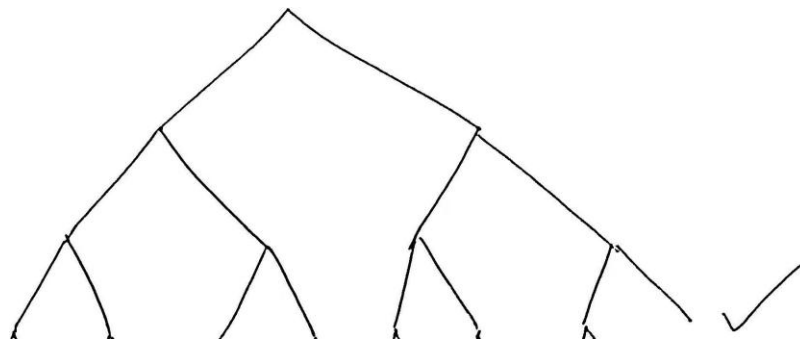
空间复杂度: $O(bd)$

迭代加深搜索可以比宽度优先搜索更高效: 不用扩展深度限制上的节点。
但是宽度优先搜索需要扩展直到目标节点。

迭代加深搜索性质



BFS



IDS

时间复杂度: $(d+1)b^0 + db + (d-1)b^2 + \dots + b^d = O(b^d)$

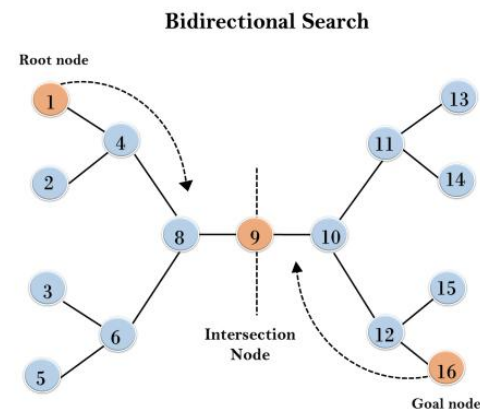
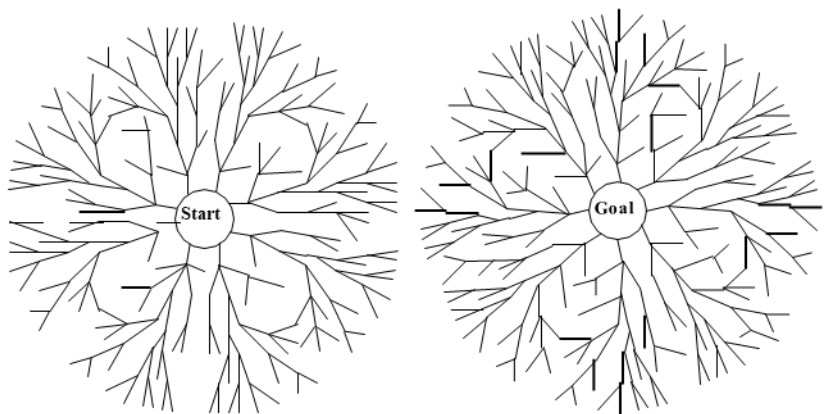
对比宽度优先搜索的时间复杂度:

$$1 + b + b^2 + \dots + b^d + b(b^d - 1) = O(b^{d+1})$$

迭代加深搜索可以比宽度优先搜索更高效: 不用扩展深度限制上的节点。
但是宽度优先搜索需要扩展直到目标节点。

空间复杂度: $O(bd)$

双向搜索



同时进行从初始状态向前的搜索和从目标节点向后搜索，
在两个搜索在中间相遇时停止搜索
假设两个搜索都使用宽度优先搜索

- 完备性: Yes
- 最优性: Yes (在每条边/每个动作的成本一致的情况下)
- 时间和空间复杂度: $O(b^{d/2})$

难点: 如何向后搜索, 部分问题可以向后搜索, 但有些问题向后搜索较难



盲目搜索总结

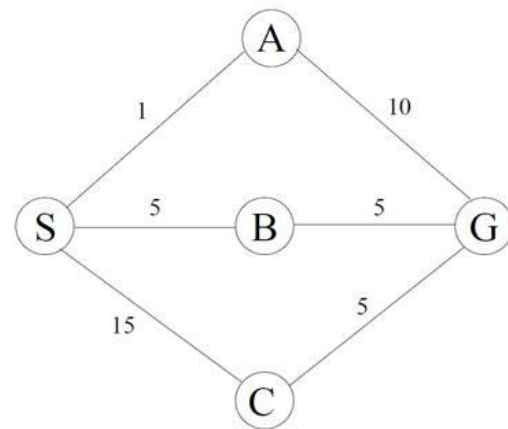
标准	深度优先	宽度优先	深度受限	迭代加深	一致代价
时间	$O(b^m)$	$O(b^{d+1})$	$O(b^L)$	$O(b^d)$	$O(b^{C^*/s+1})$
空间	$O(bm)$	$O(b^{d+1})$	$O(bL)$	$O(bd)$	$O(b^{C^*/s+1})$
最优	否	是	否	是	是
完备	否	是	否	是	是

上表中， b 为问题中一个状态最大的后继状态个数， d 是最短解的动作个数， m 是遍历过程中最长路径的长度， L 为限制的搜索深度， C^* 为最优解的成本， s 为动作的成本下界。

课堂练习

练习1 如右图所示，初始状态为节点S，目标状态为G，各边的路径成本如图。请使用一致代价搜索算法，完成以下任务：

- 按扩展顺序列出节点，并记录每一步的Open表和Closed表状态



课堂练习

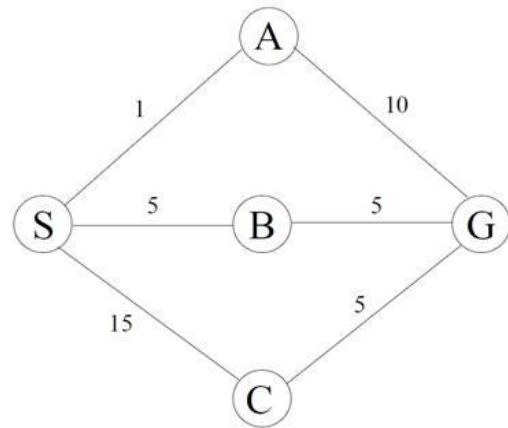
练习1 解:

- 搜索过程如下: (括号内为该节点总代价)

步骤	扩展节点	Open表	Closed表	备注
0	-	S(0)	空	初始状态
1	S	A(1), B(5), C(15)	S(0)	扩展S, 生成子节点A/B/C
2	A	B(5), G(11), C(15)	S(0), A(1)	扩展A, 生成G (总成本 1+10=11)
3	B	G(10), C(15)	S(0), A(1), B(5)	扩展B, 生成G (总成本 5+5=10)
4	G	C(15)	S(0), A(1), B(5), G(10)	找到目标G, 终止

最终路径为 $S \rightarrow B \rightarrow G$, 总成本为 10

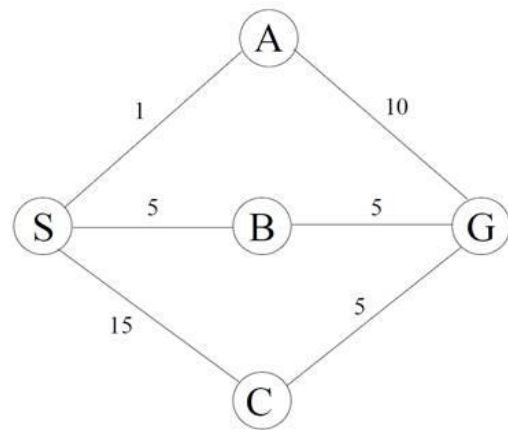
当节点A生成目标G时, 为何算法不立即终止? 解释UCS如何保证最终找到最优路径。



课堂练习

练习1 解:

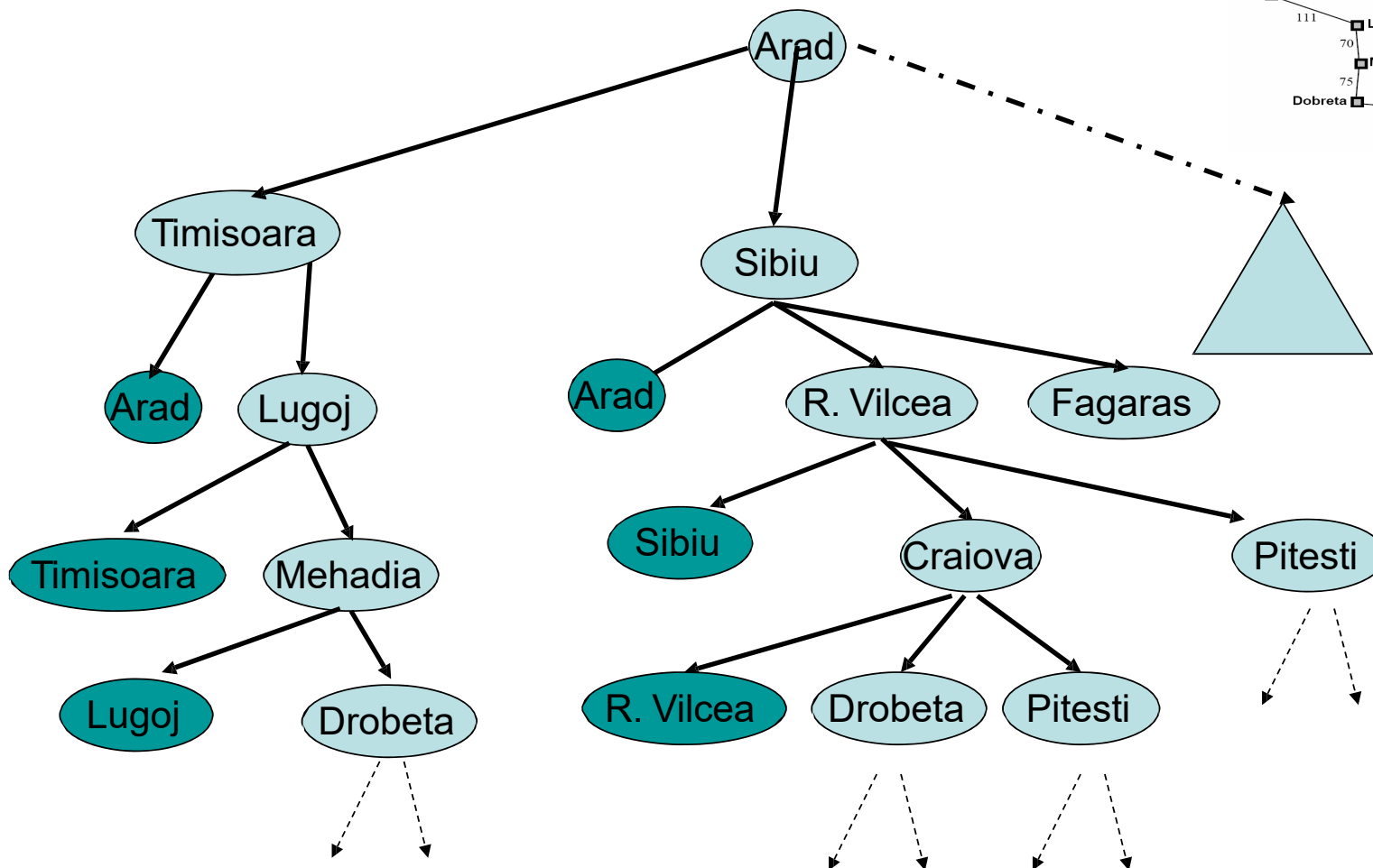
- 当节点A生成目标G（路径成本11）时，算法未终止的原因：
 - UCS仅在扩展节点时检查是否为目标，而非生成时
 - 此时Open表中存在B(5)，其潜在路径到G的总成本可能更低（实际为 $5+5=10$ ）
 - UCS始终优先扩展累积成本最小的节点，确保首次到达目标时路径成本最小。若提前终止，可能错过更优路径（ $B \rightarrow G$ ）。





路径检测

- 回顾下，我们之前在边界上通常保存了路径
- 假设 $(n_1, \dots, n_k)$ 是一条到达节点 n_k 的路径，并且我们要扩展节点 n_k 来获得子节点 c ，我们可以获得一条到达节点 c 的路径 $(n_1, \dots, n_k, c)$
- 路径检测用于确保状态（节点） c 与它所在路径上的祖先节点都不相等
- 也就是说，单独检测每条路径是否出现重复节点

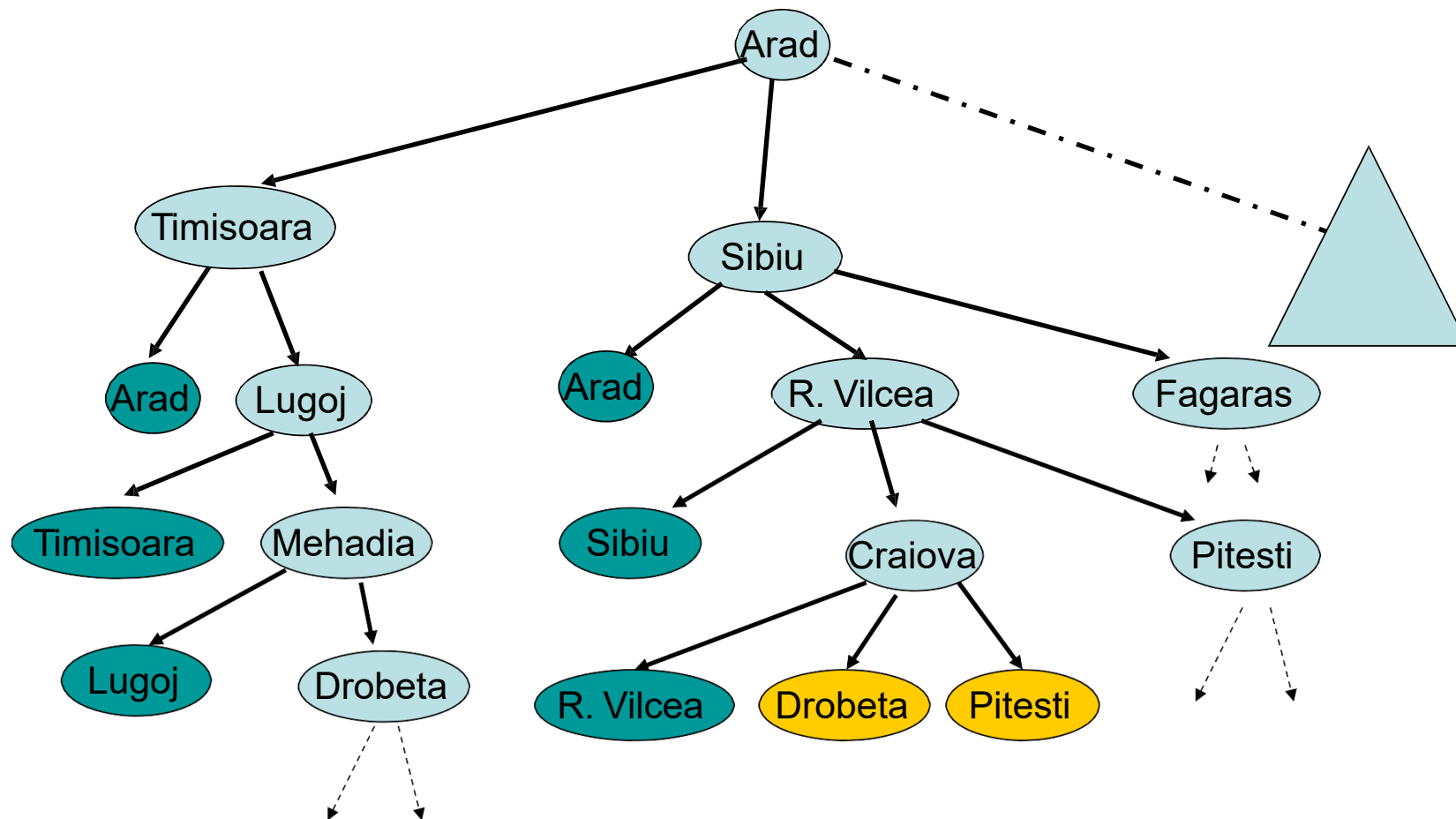




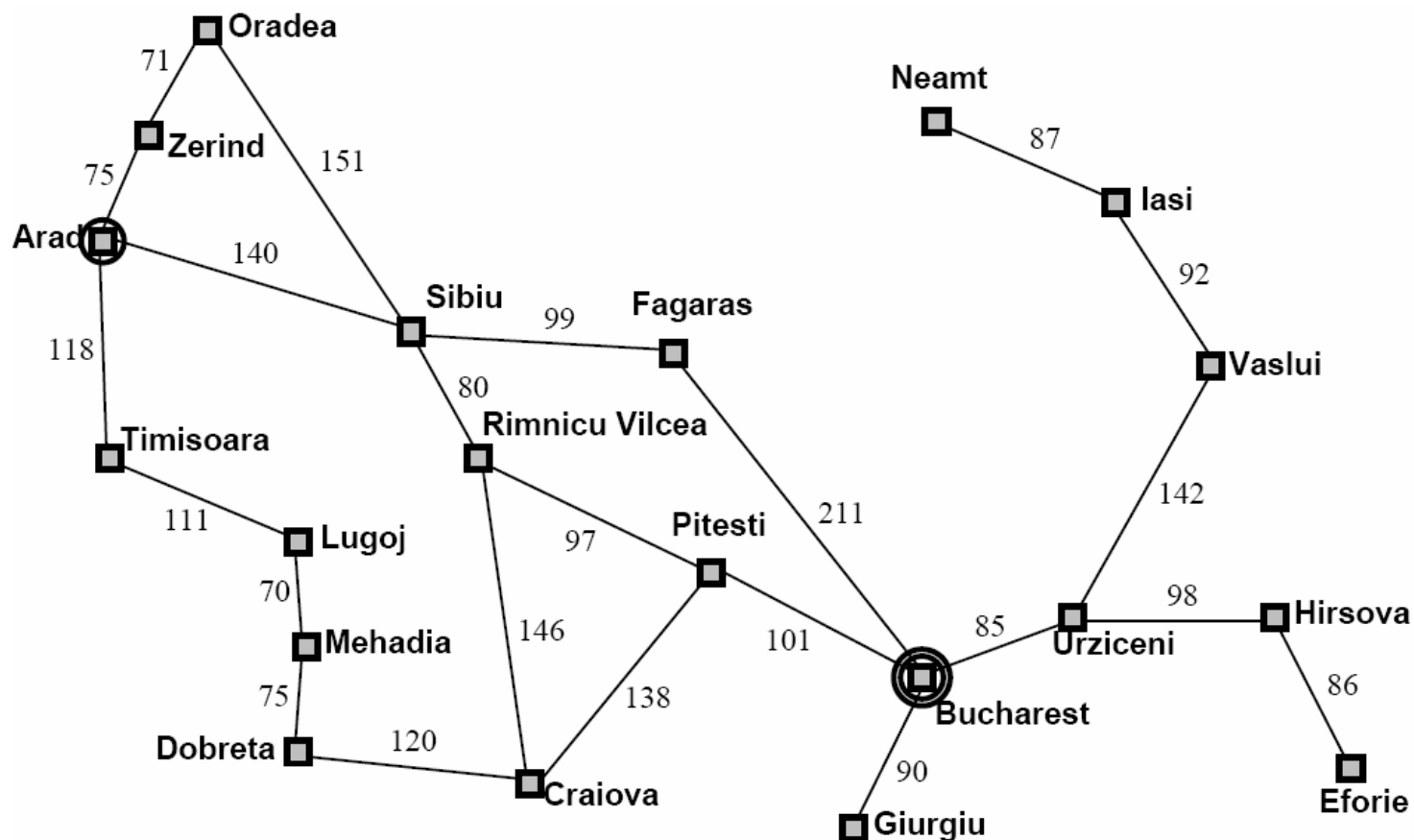
Cycle Checking

- Keep track of **all states** previously expanded during the search.
记录下在之前的搜索过程中扩展过的所有节点
- 当扩展节点 n_k 获得子节点 c 时, 确保节点 c 不等于之前任何扩展过的节点
- Higher space complexity (equal to the space complexity of breadth-first search).
- Other issues with cycle checking will come up when we look at heuristic search.

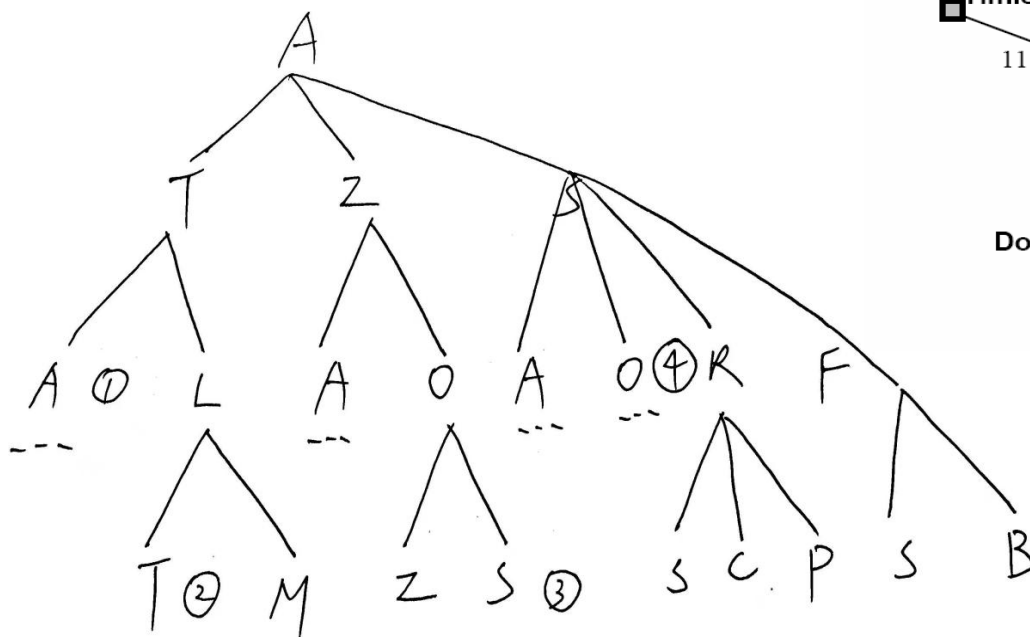
Cycle Checking (BFS)



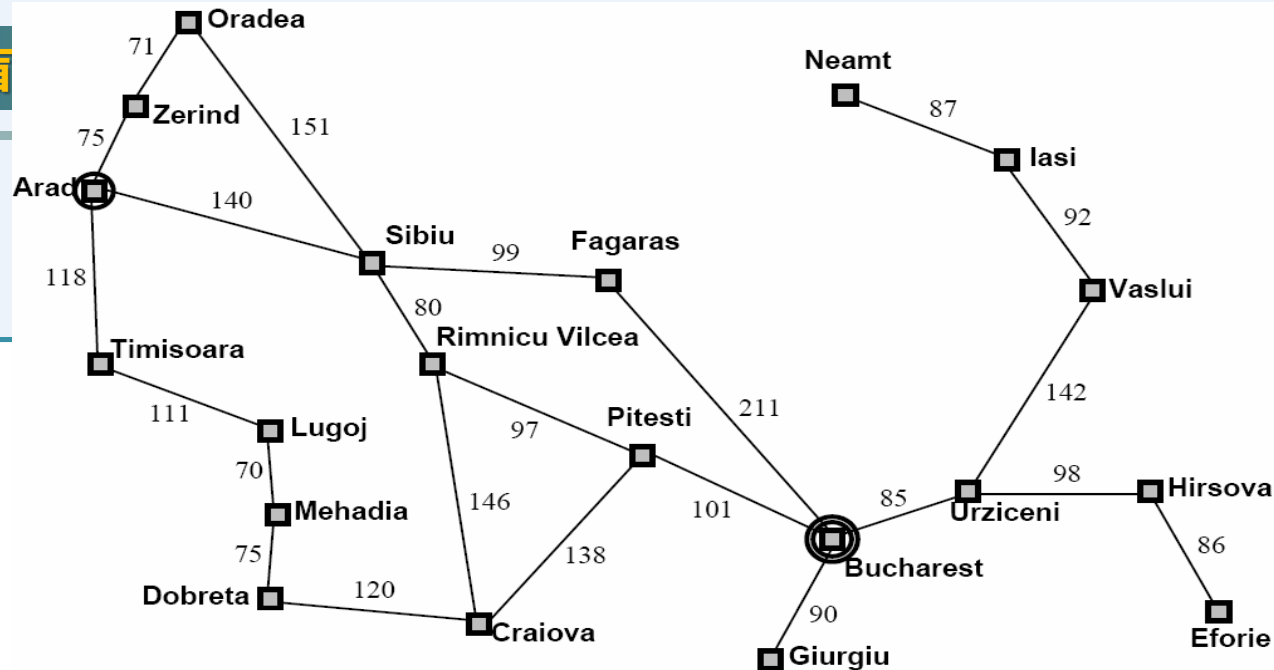
Example: Arad to Neamt



Example: Arad to Neamt



- If path checking, nodes 1 and 2 are not generated
- If cycle checking, node 3 is not generated since it is expanded before; but if only path checking, node 3 is generated
- If cycle checking, node 4 is generated, because it is only **generated** before, not **expanded** before





环检测最优性问题

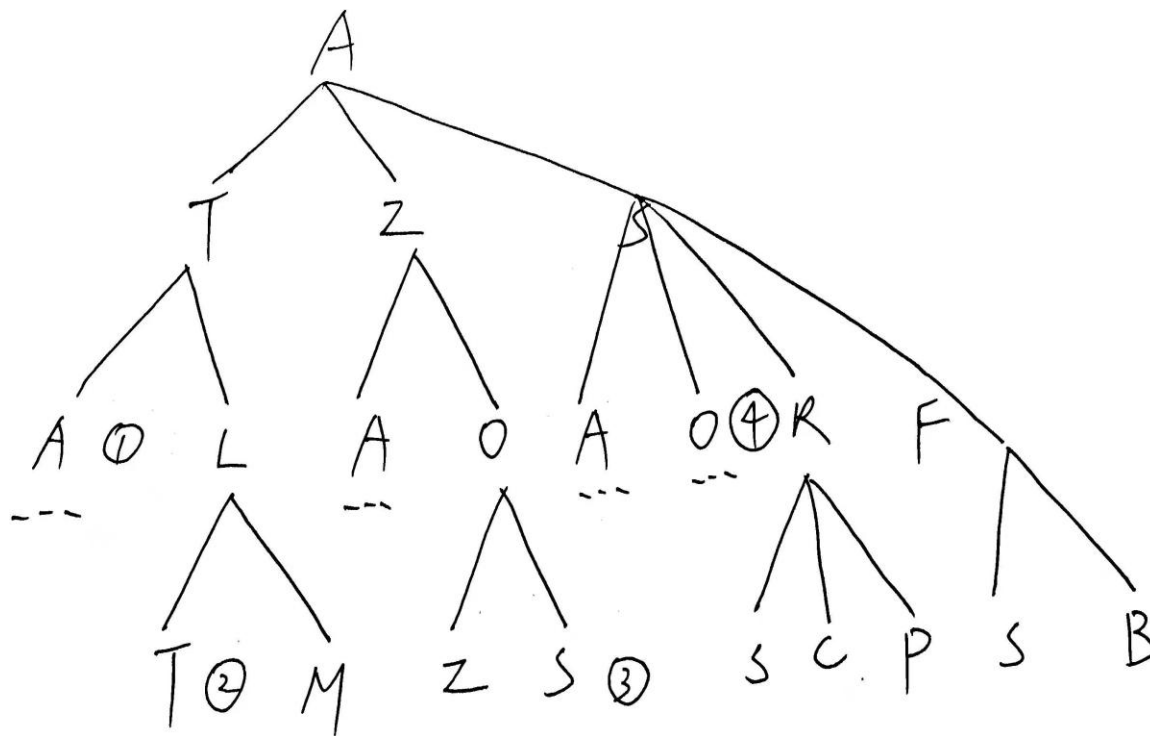
对于一致代价搜索，使用环检测后仍能找到最优的解

- 一致代价搜索在第一次扩展到某个节点时，其实已经找到了到达这个节点的成本最低的路径
- 这意味着被环检测剔除的节点不可能出现一条更短/成本更低的路径



环检测最优性问题

之后会看到，对于启发式搜索，这个性质不一定会成立



e.g., on the previous slide, when we expand the first O to generate node 3, S is already expanded, so $c(A \rightarrow S) \leq C(A \rightarrow Z \rightarrow O)$. Thus node 3 can be safely rejected.



路径/环检测 比较

路径检测: 当扩展节点 n 来获得子节点 c 时, 确保节点 c 不等于到达节点 c 的路径上的任何祖先节点

环检测: 记录下在之前的搜索过程中扩展过的所有节点
当扩展节点 n_k 获得子节点 c 时, 确保节点 c 不等于之前任何扩展过的节点

对于一致代价搜索, 环检测可以保留一致代价搜索的最优性